BIT.MLMR

Machine Learning



Lecture 2

Fritz Perls: Learning is the discovery that something is possible

Herbert Simon: Learning denotes change







Outline

- **What and Why do machines learn?**
- **Supervised Learning**
  - Rote Learning
  - Decision Trees
  - Learning in Bayesian Inference
- **Unsupervised Learning**
  - Partition clustering
  - Hierarchical clustering
  - Density based clustering
  - Grid based clustering

PartI: What and Why do machines learn?

What do machines learn?

- ***Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the task or tasks drawn from the same population more efficiently and more effectively the next time.***

  by Herbert Simon

- ***Machine Learning is the study of computer algorithms that improve automatically through experience. Applications range from data mining programs that discover general rules in large data sets, to information filtering systems that automatically learn users' interests.***

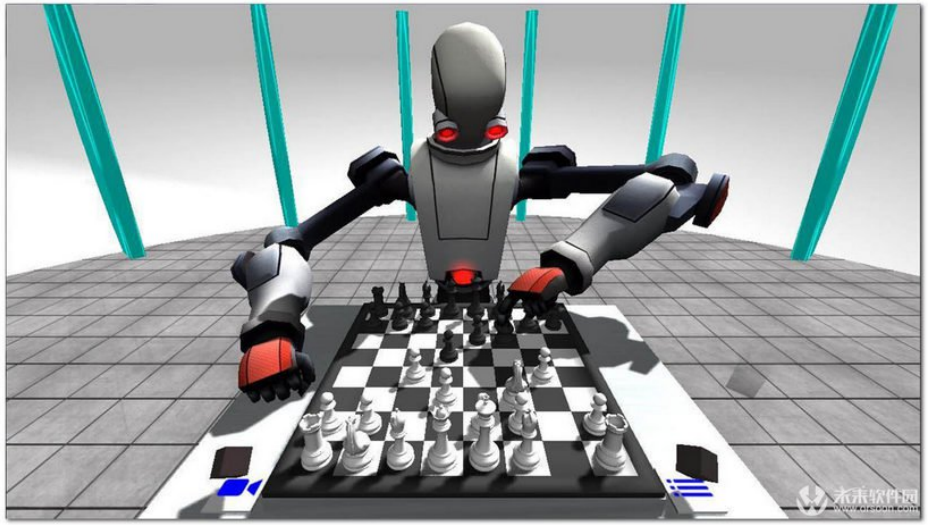
  by Tom M. Mitchell

Defining the Learning Task

Improve on Task with respect to Performance metric based on Experience

Examples

T: Playing checkers
P: Percentage of games won against an arbitrary opponent
E: Playing practice games against itself

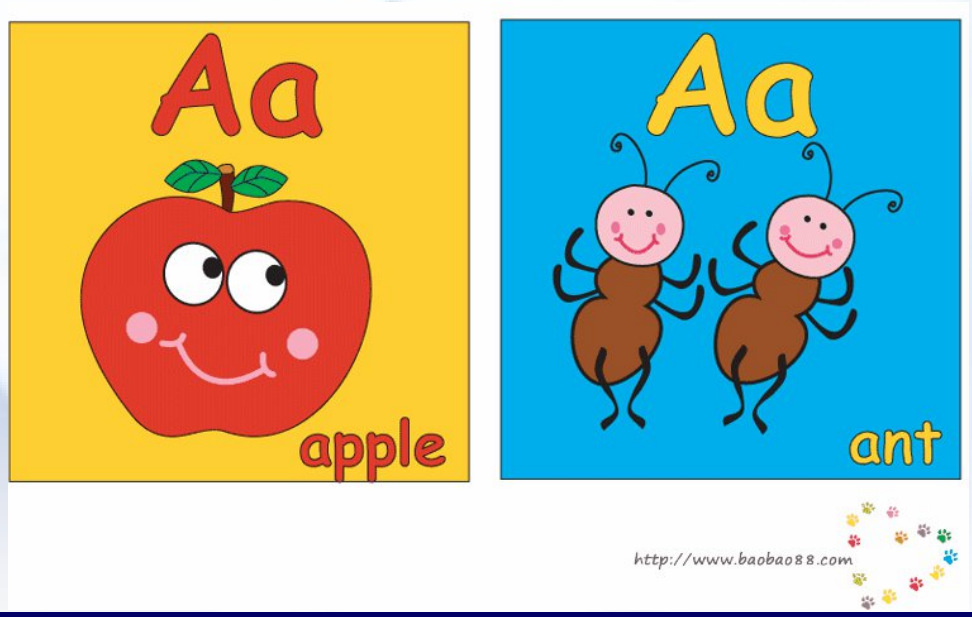


Defining the Learning Task

Improve on Task with respect to Performance metric based on Experience

Examples

T: Recognizing hand-written words
P: Percentage of words correctly classified
E: Database of human-labeled images of handwritten words



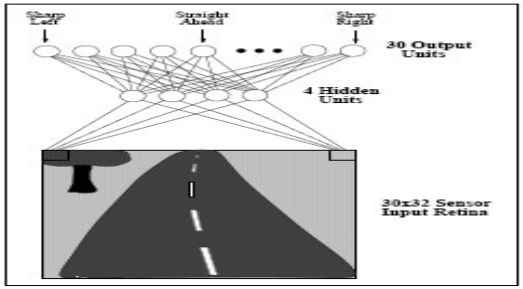
Defining the Learning Task

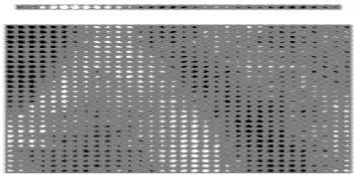
Improve on Task with respect to Performance metric based on Experience

Examples

T: Driving on four-lane highways using vision sensors
P: Average distance traveled before a human-judged error
E: A sequence of images and steering commands recorded while observing a human driver.







Definition of Learning

Learning is a process through which the subject gains knowledge and increase his abilities. The inner behavior in the learning is gaining knowledge, accumulating experiences, and discovering rules; the outer manifestation of it is improving performance, fitting environment, and achieving self-perfection.

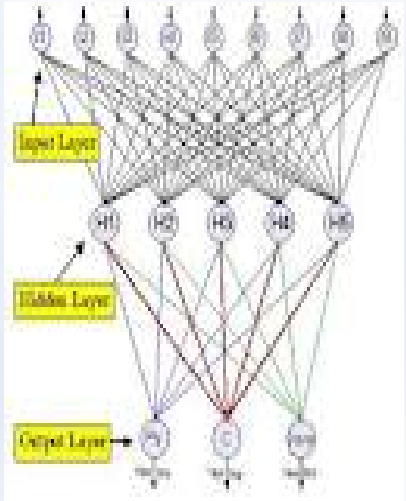
The results of machine learning are embodied in two aspects: effectiveness and efficiency of the computation.

Why do machines learn?

- **Learning is essential for unknown environments (Develop systems that can automatically adapt and customize themselves).**
  - i.e., when designers lacks omniscience

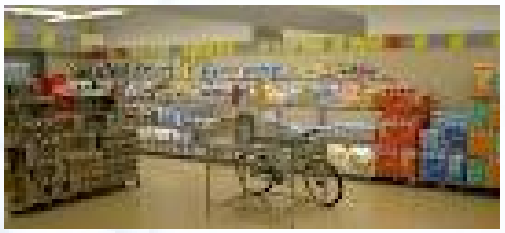
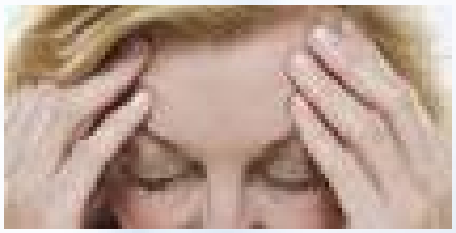


- **Learning is useful as a system construction method (Develop systems that are too difficult/expensive to construct manually).**
  - i.e., expose the system to reality rather than trying to write it down



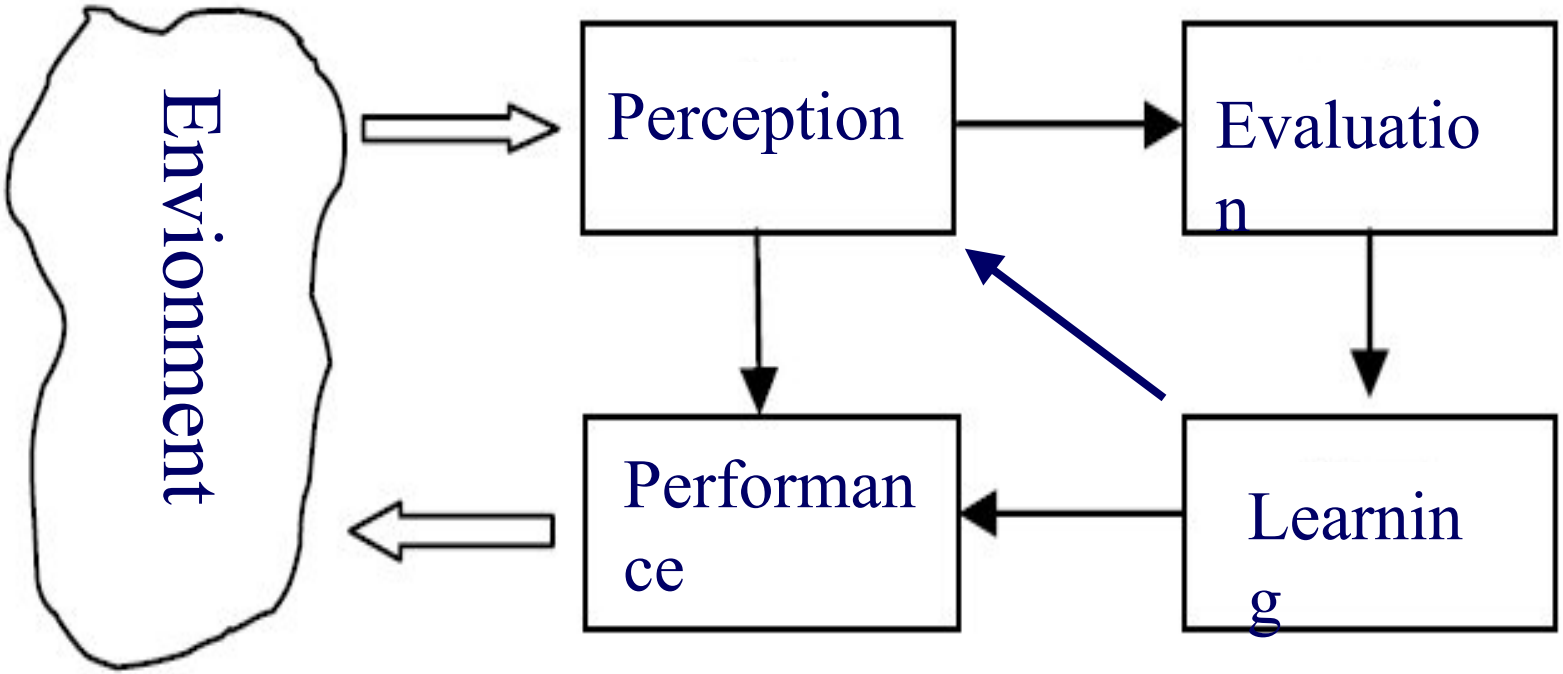
Why do Machines Learn? contd.

- **Discover new knowledge from large databases (*data mining*).**
  - Market basket analysis (e.g. diapers and beer)
  - Medical text mining (e.g. migraines to calcium)

- **Computational studies of learning may help us understand learning in humans and other biological organisms.**

How do machines learn?



Learning element

- **Two key issues**
  - What is the best machine? Objectiveness
  - How to achieve the best machine? Optimization
- **Type of feedback:**
  - Supervised learning: correct answers for each example
  - Unsupervised learning: correct answers not given
  - Reinforcement learning: occasional rewards

Part II : Supervised Learning

2.1 The simplest form of Supervised Learning: Rote Learning

- Learning == Memory?
- Save solutions to problems and retrieve them when encountering the same problem.
- Trade-off between costs of saving and computing.

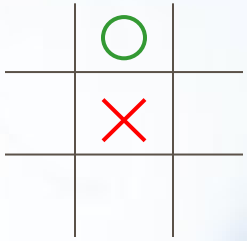
Rote Learning Example: Checker Playing program of Samuel



- **Performance element**
  Max-Min search

MiniMax Example: tic-tac-toe

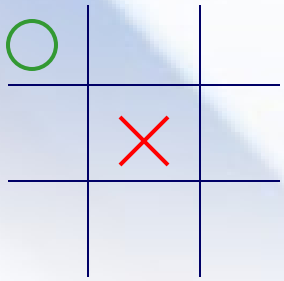
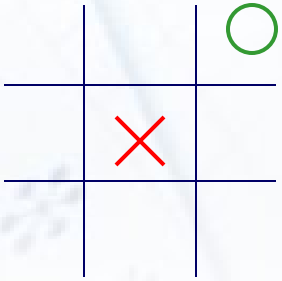
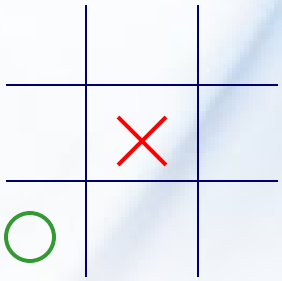
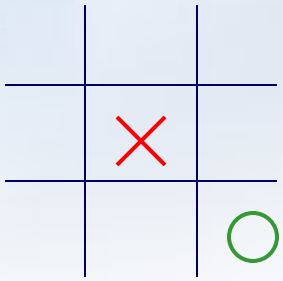
- **Evaluation function:**



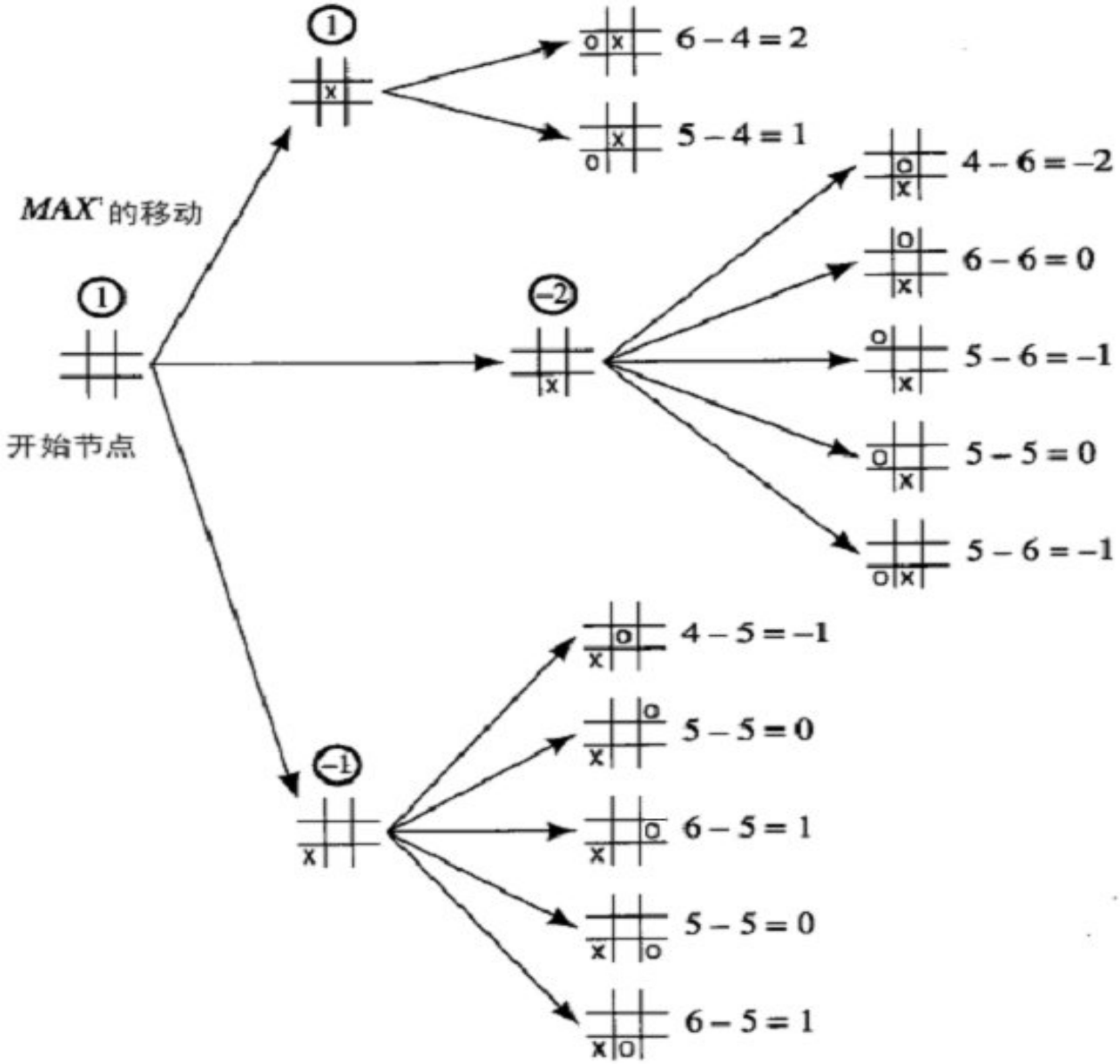
e(P) = 6 - 4 = 2

$$e(P) = e(+P) - e(-P)$$

- **Reserving only one for the symmetrical states**

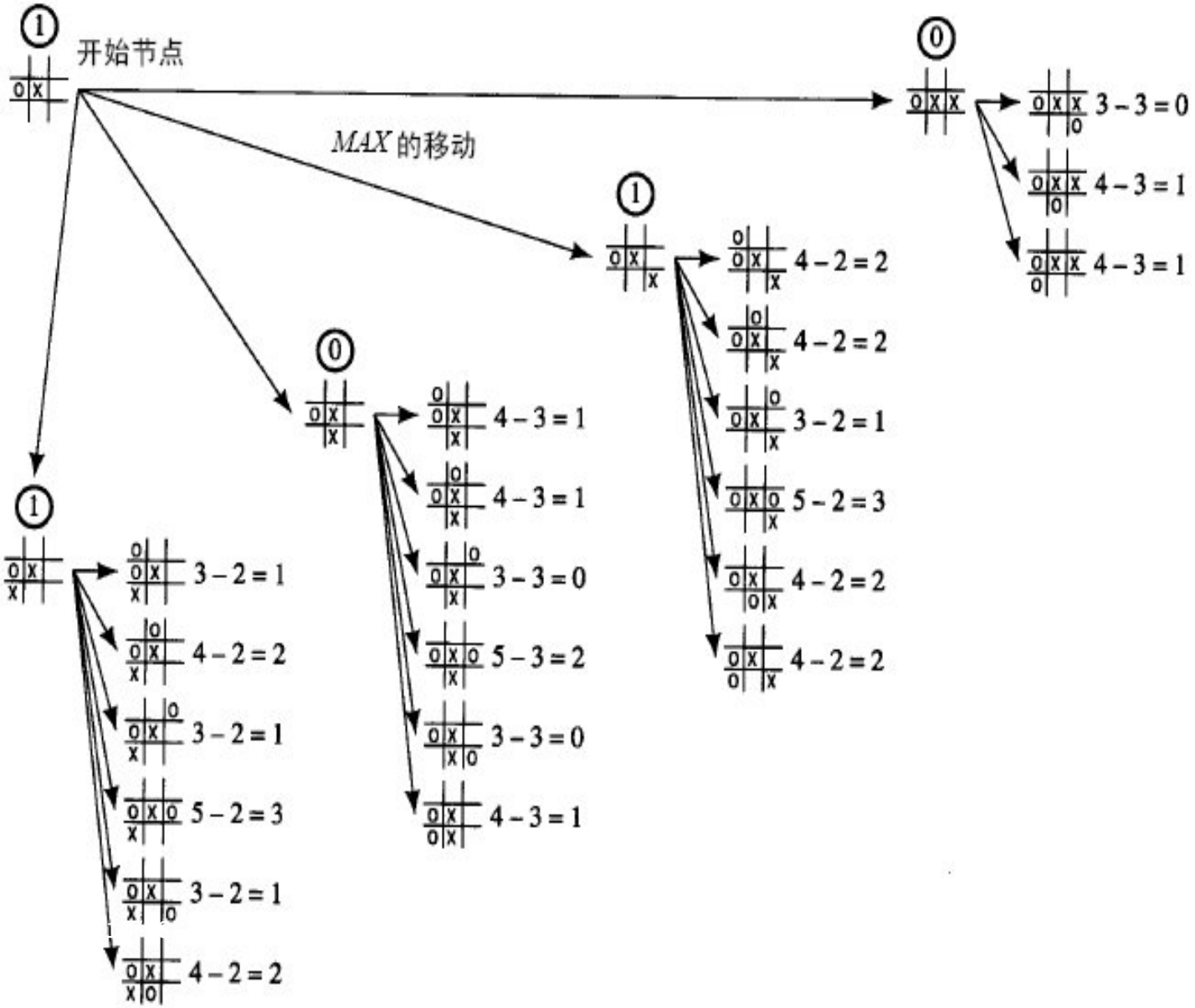





Solution of tic-tac-toe (1)



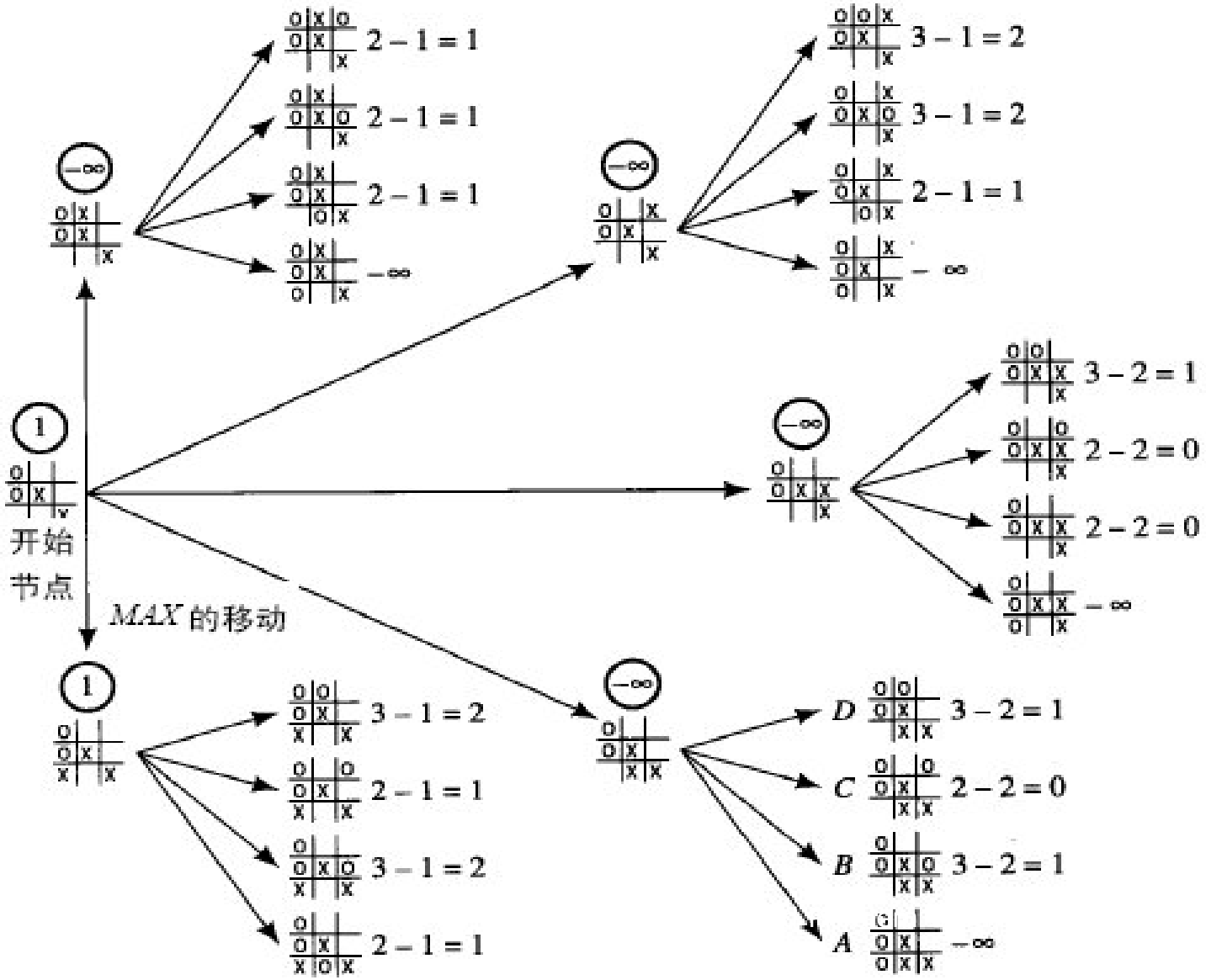
First stage

Solution of tic-tac-toe (2)



Second stage

Solution of tic-tac-toe (3)



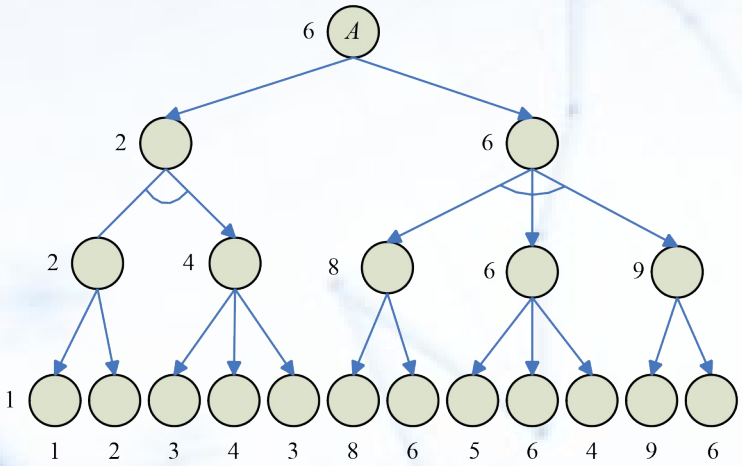
Final stage

Checker Playing program of Samuel

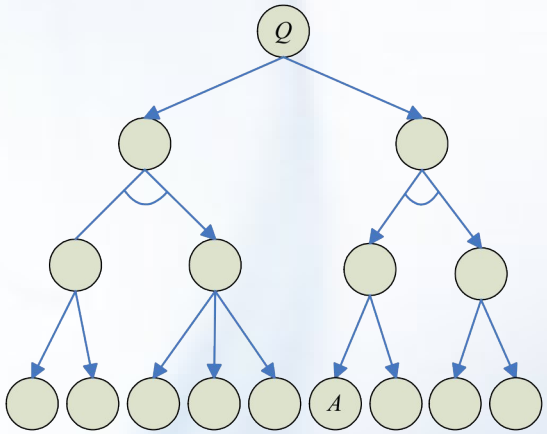
- **Learning element**

  What is saved?

  the pair of state -- minimax value



Save



Retrieve

Why more intelligent?

increase the search depth of checker program

2.2 What is Supervised Learning?

- **Learn a function from examples**

  ***f* is the target function**

  An example is a pair (*x*, *f*(*x*))

  Problem: find a hypothesis *h*

  such that $h \approx f$

  given a training set of examples

Supervised Learning

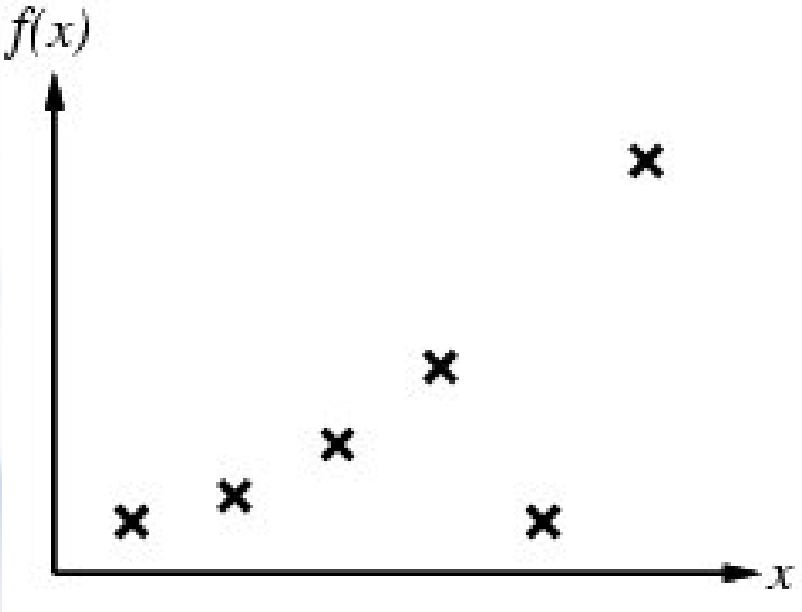
- **Two key problems:**
  - **How to define the function form?**
  - **How to optimize the function parameters or even its structure?**

Supervised learning principle

- Construct/adjust *h* to agree with *f* on training set

  (*h* is consistent if it agrees with *f* on all examples)

- E.g., curve fitting:

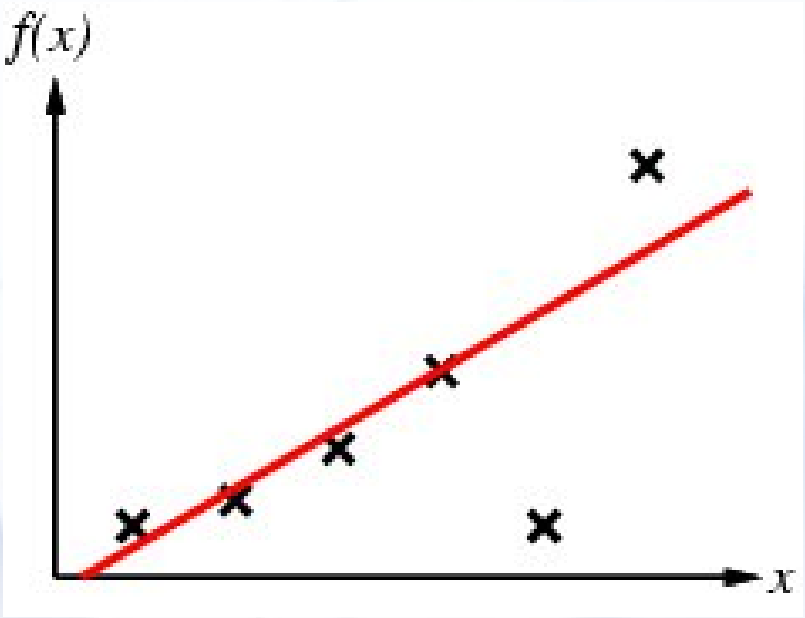


Supervised learning principle

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



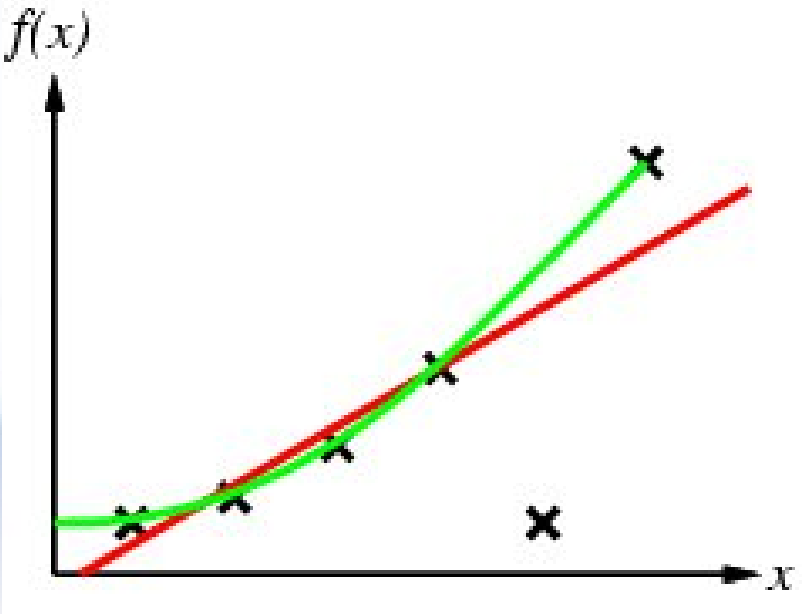
h_1: $y = ax + b$

Supervised learning method

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



h_1: $y = ax + b$

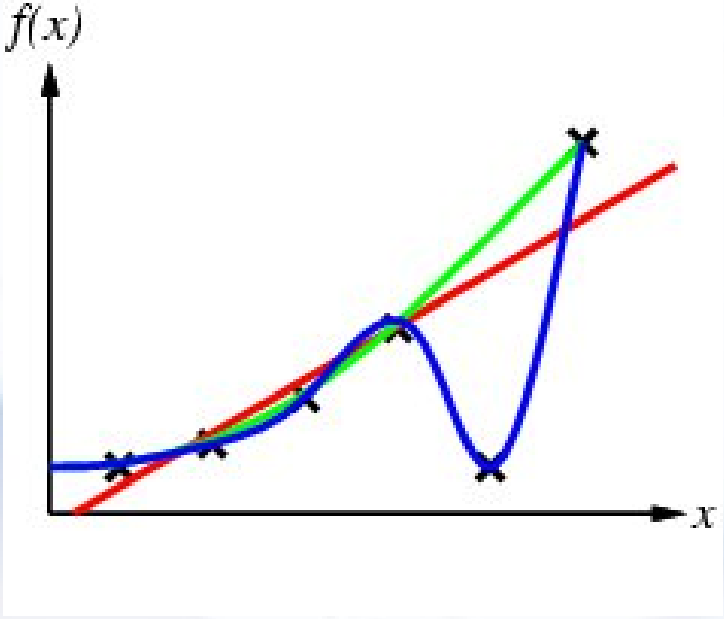
h_2: $y = ax^2 + bx + c$

Supervised learning method

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



h_1: $y = ax + b$

h_2: $y = ax^2 + bx + c$

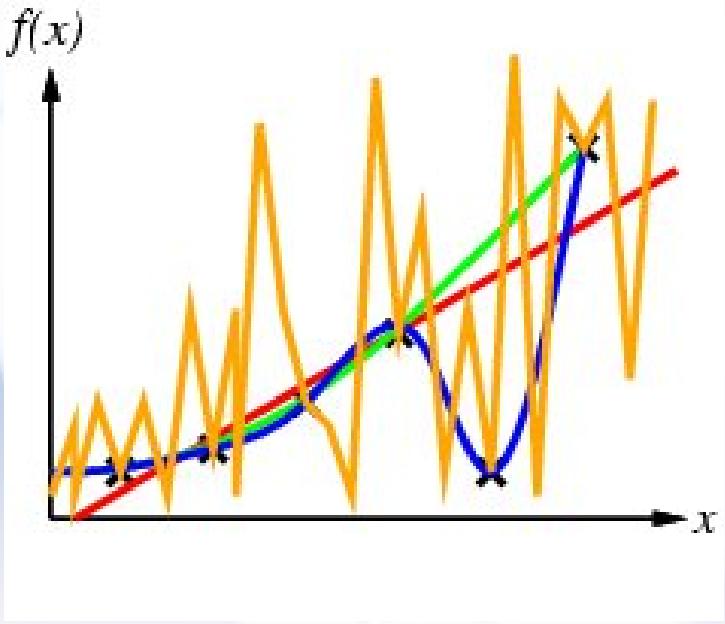
h_3: $y = ax^4 + bx^2 + cx + d$

Supervised learning method

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



h_1: $y = ax + b$

h_2: $y = ax^2 + bx + c$

h_3: $y = ax^4 + bx^2 + cx + d$

h_4: *too complicated*

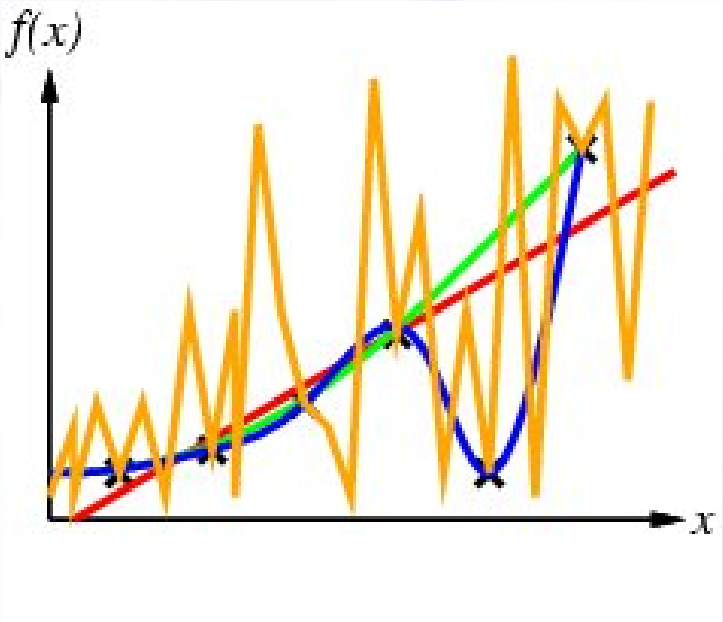
Supervised learning method

- Construct/adjust *h* to agree with *f* on training set

  (*h* is consistent if it agrees with *f* on all examples)

- E.g., curve fitting:





- Ockham's razor: prefer the simplest hypothesis consistent with data

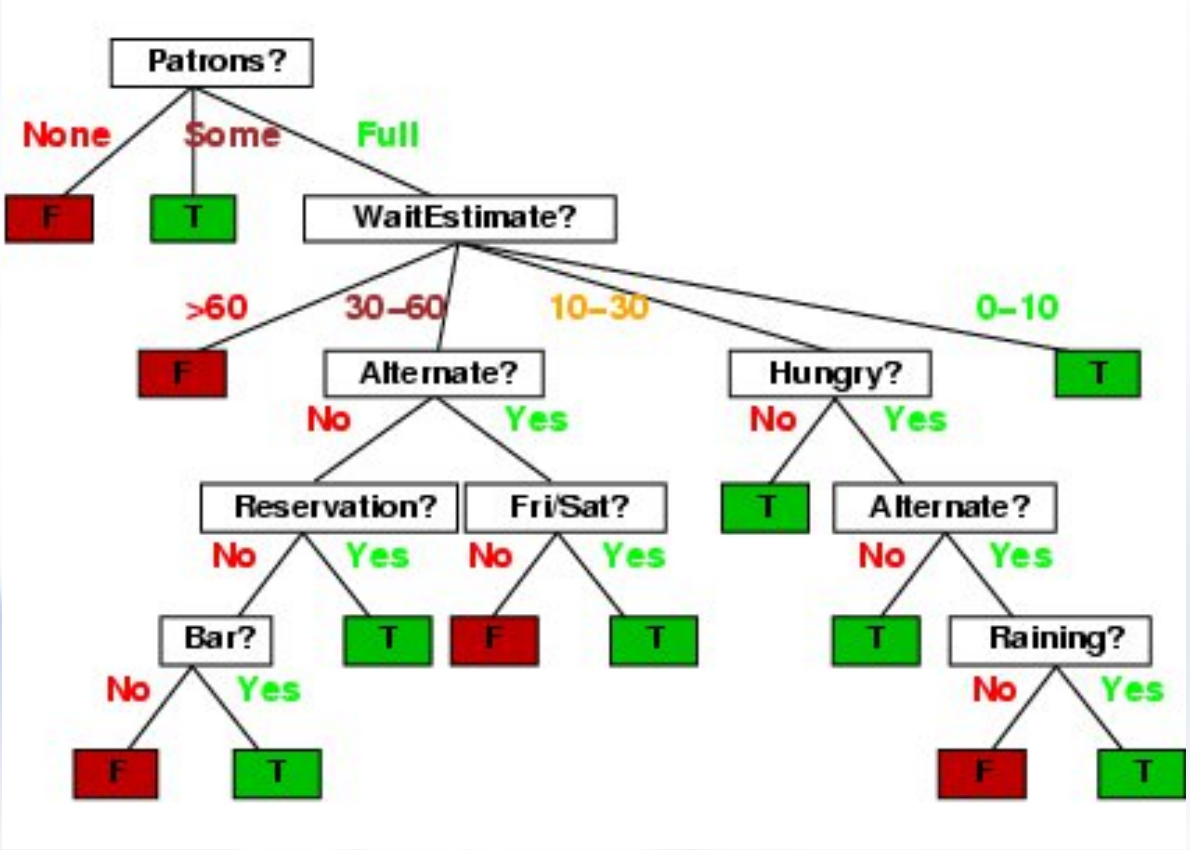
2.3 Decision trees and Learning

- **A Demo Problem: decide whether to wait for a table at a restaurant, based on the following attributes:**
  1. Alternate: is there an alternative restaurant nearby?
  2. Bar: is there a comfortable bar area to wait in?
  3. Fri/Sat: is today Friday or Saturday?
  4. Hungry: are we hungry?
  5. Patrons: number of people in the restaurant (None, Some, Full)
  6. Price: price range ($, $$, $$$)
  7. Raining: is it raining outside?
  8. Reservation: have we made a reservation?
  9. Type: kind of restaurant (French, Italian, Thai, Burger)
  10. WaitEstimate: estimated waiting time (0-10, 10-30, 30-60, >60)

Problem from Stuart J. Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*

Decision tree example

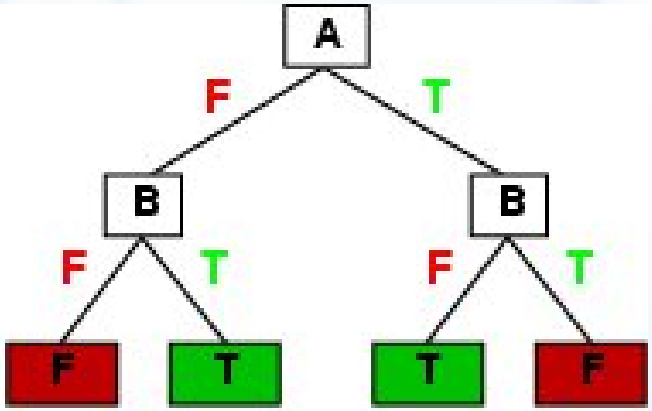
- One possible representation for hypotheses
- E.g., here is the "true" tree for deciding whether to wait:



Expressiveness

- Decision trees can express any function of the input attributes. E.g., for Boolean functions, truth table row → path to leaf:

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



- How to construct decision tree in applications, such as in restaurant problem?

Learning by experience

Training Data for Restaurant Problem

- **Examples described by attribute values (Boolean, discrete, continuous), i.e, situations where I will/won't wait for a table:**

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*Wait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

- **Problem is transformed into "Classifying examples into positive (T) or negative (F)"**

Hypothesis Spaces

- Trivially, there is a consistent decision tree for any training set with one path to leaf for each example (unless *f* nondeterministic in *x*) but it probably won't generalize to new examples.

  How many distinct decision trees with *n* Boolean attributes?

- Learning Task : Find out the optimal DT from these distinct DTs based on the training data.

  Prefer to find more compact decision trees

Decision tree learning

- **Aim**

  Find a small tree consistent with the training examples

- **Idea**

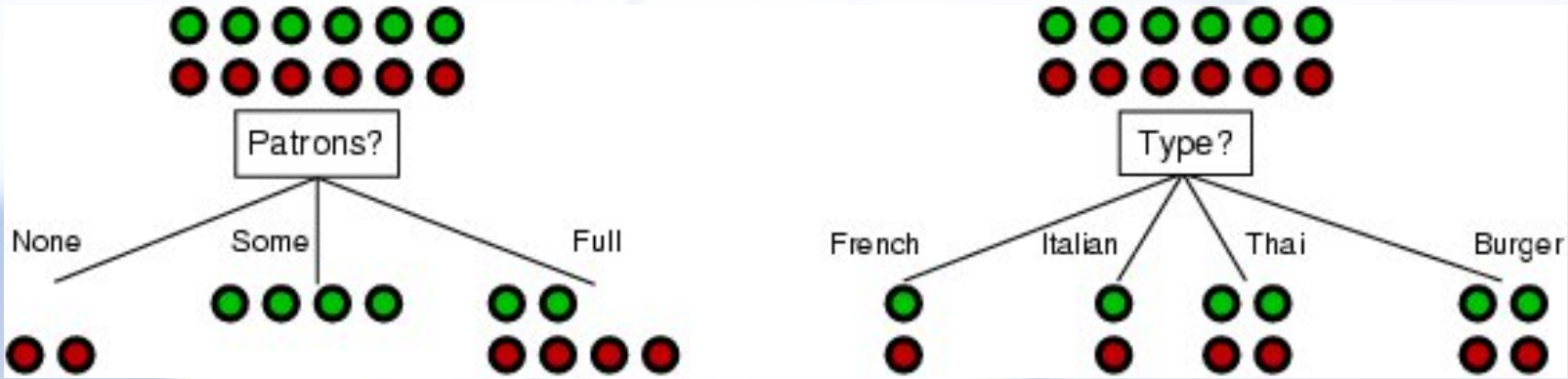
  (recursively) Choose "most significant" attribute as root of (sub)tree

Decision tree learning

```
function DTL(examples, attributes, default) returns a decision tree
    if examples is empty then return default
    else if all examples have the same classification then return the classification
    else if attributes is empty then return MODE(examples)
    else
        best ← CHOOSE-ATTRIBUTE(attributes, examples)
        tree ← a new decision tree with root test best
        for each value v_i of best do
            examples_i ← {elements of examples with best = v_i}
            subtree ← DTL(examples_i, attributes − best, MODE(examples))
            add a branch to tree with label v_i and subtree subtree
        return tree
```

Choosing an attribute

- **Idea: a good attribute splits the examples into subsets that are (ideally) "all positive" or "all negative"**



- ***Patrons?*** **is a better choice** Why?

Using information theory

- To implement `Choose-Attribute` in the DTL algorithm
- Information Content (Entropy):

$I(P(v_1), \ldots, P(v_n)) = \Sigma_i -P(v_i) \log_2 P(v_i)$

- For a training set containing *p* positive examples and *n* negative examples:

$$I(\frac{p}{p+n}, \frac{n}{p+n}) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

Information gain

- A chosen attribute *A* divides the training set *E* into subsets $E_1, \ldots, E_v$ according to their values for *A*, where *A* has *v* distinct values.

$$remainder\ (A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p + n} I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right)$$

- **Information Gain (IG)** or reduction in entropy from the attribute test:

$$IG(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - remainder(A)$$

- Choose the attribute with the **largest IG**

Information gain

- **For the training set**

$$p = n = 6,\ I(6/12, 6/12) = 1$$

- **Consider the attributes *Patrons* and *Type* (and others too):**

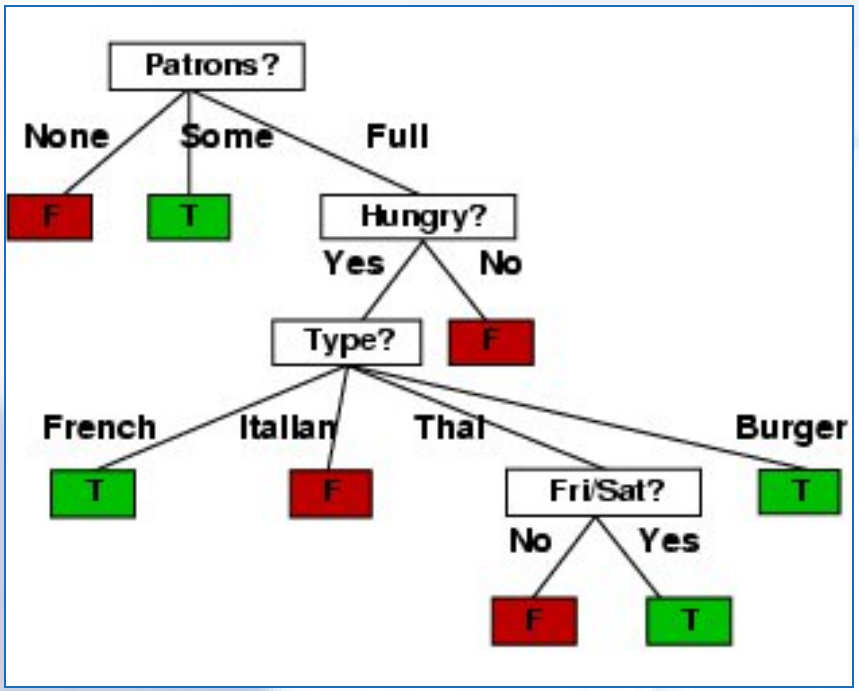
$$IG(Patrons) = 1 - \left[\frac{2}{12} I(0,1) + \frac{4}{12} I(1,0) + \frac{6}{12} I\left(\frac{2}{6}, \frac{4}{6}\right)\right] = .0541 \text{ bits}$$

$$IG(Type) = 1 - \left[\frac{2}{12} I\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12} I\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12} I\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{4}{12} I\left(\frac{2}{4}, \frac{2}{4}\right)\right] = 0 \text{ bits}$$

In fact, *Patrons* has the highest IG of all attributes and so is chosen by the DTL algorithm as the root

Example contd.

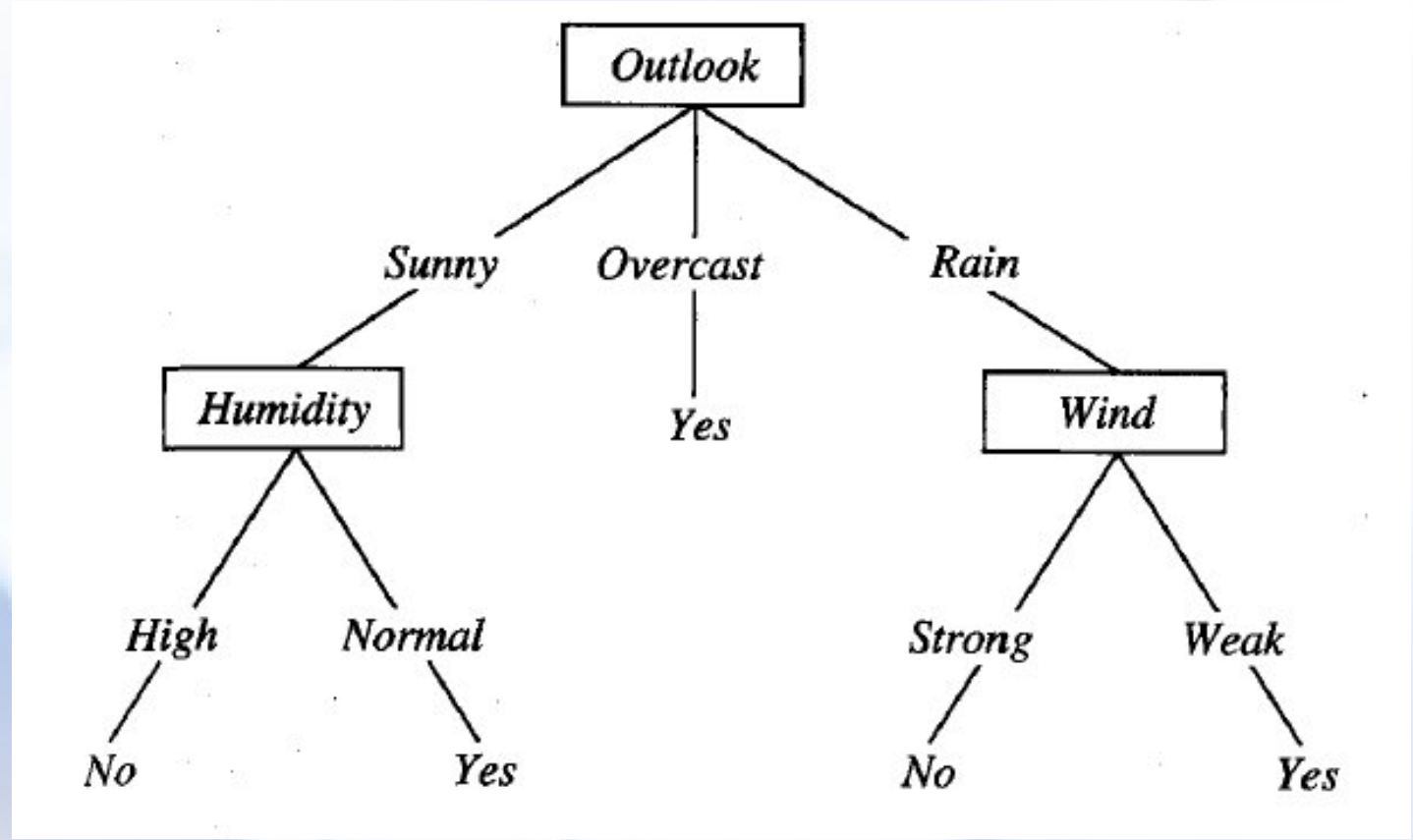
- Decision tree learned from the 12 examples:



Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*Wait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

- Substantially simpler than “true” tree---a more complex hypothesis isn’t justified by small amount of data

Example2: Learning the concept *PlayTennis*



Example 2 From Tom M. Mitchell, *Machine Learning*

Training Examples

Day	*Outlook*	*Temperature*	*Humidity*	*Wind*	*PlayTennis*
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Choose the root attribute

$$Entropy(S) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940$$

$$Entropy(S, Wind) = \sum_{v \in \{Weak, Strong\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

S_{weak} : 6 yes, 2 no; S_{strong} : 3 yes, 3 no;

$$Entropy(S, Wind) = -\frac{8}{14} Entropy(S_{weak}) - \frac{6}{14} Entropy(S_{strong})$$

$$= -\frac{8}{14} 0.811 - \frac{6}{14} 1.00 = 0.892$$

Choose the root attribute

$$Gain(S, Wind) = 0.940 - 0.892 = 0.048$$

Similarly,

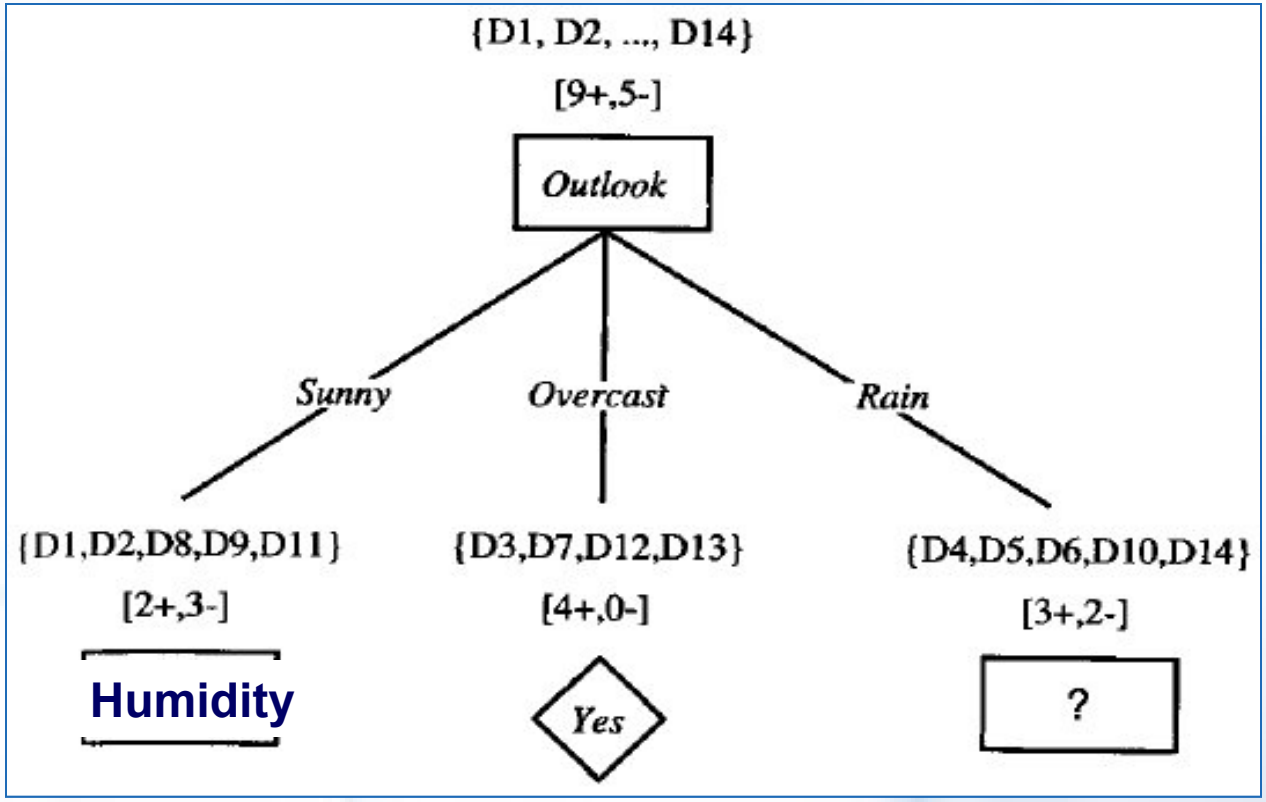
$$Gain(S, Outlook) = 0.246$$

$$Gain(S, Humidity) = 0.151$$

$$Gain(S, Temperature) = 0.029$$

Winner: Outlook

Tree generation illustration



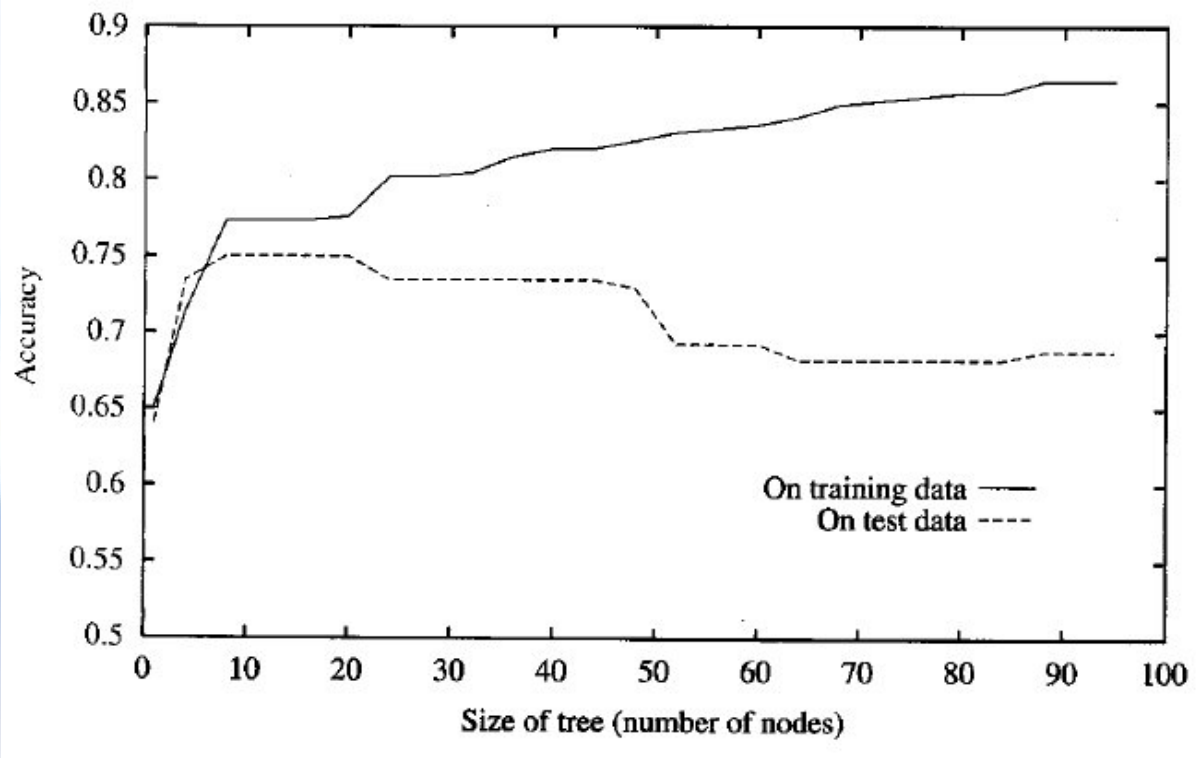
$$Gain\ (S_{sunny}, Humidity) = .970 - (3/5)\ 0.0 - (2/5)\ 0.0 = .970$$

$$Gain\ (S_{sunny}, Temperature) = .970 - (2/5)\ 0.0 - (2/5)\ 1.0 - (1/5)\ 0.0 = .570$$

$$Gain\ (S_{sunny}, Wind) = .970 - (2/5)\ 1.0 - (3/5)\ .918 = .019$$

Overfitting

- **Some other hypothesis that fits the training examples less well actually performs better over the entire distribution of instances.**



Pic Tom M. Mitchell, *Machine Learning*

How to avoid overfitting

- **Stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data**
- **Allow the tree to overfit the data, and then post-prune the tree.**

Ockham's razor

Minimum Description Length (MDL) Criterion

- **Occam's razor**

  "Prefer the simplest hypothesis that fit the data"

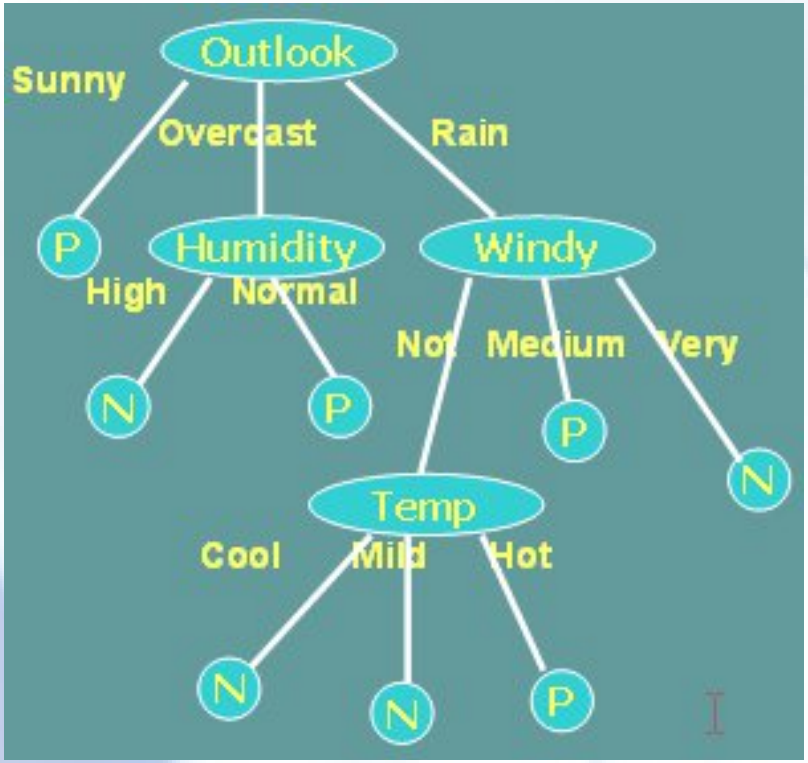
  "choose the shortest explanation for the observed data"

- **Explain the train data under decision tree. The tree with the shortest explanation is the best one**

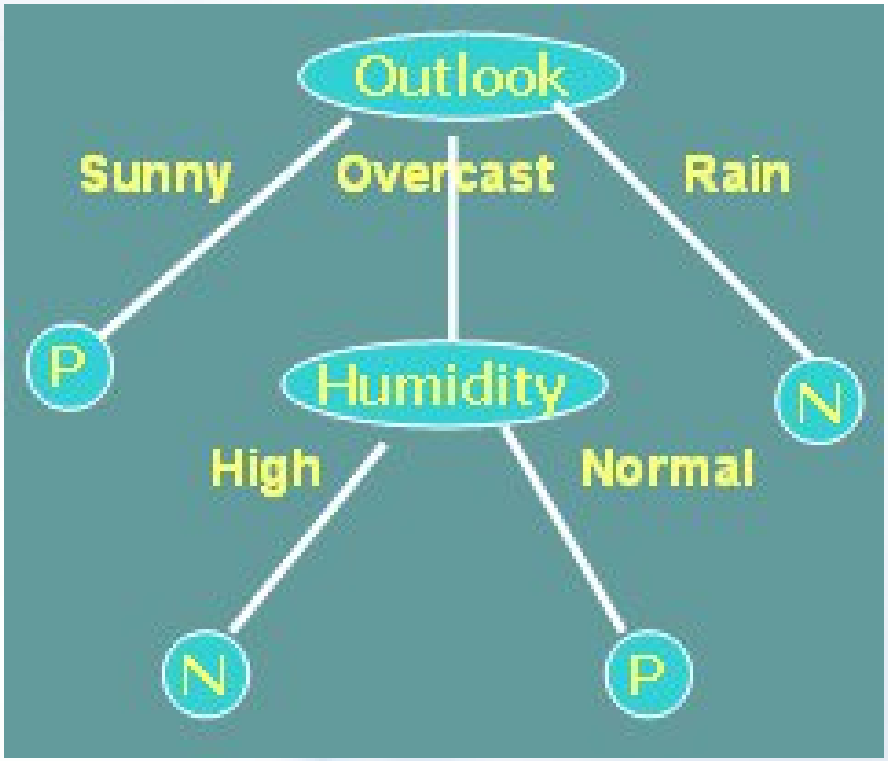
An example of MDL criterion

Attributes	Outlook	Temperature	Humidity	Windy	Class
1	Overcast	Hot	High	Not	N
2	Sunny	Mild	Normal	Very	P
8	Rain	Hot	High	Medium	P
24	Rain	Mild	High	Very	N
24 records	3 values	3 values	2 values	3 values	2 classes

An example of MDL criterion



With noise data 8



Without noise data 8

How to explain data under them?

What are their description lengths?

Description length

- DL = size of the hypothesis and any additional cost of encoding the data given this hypothesis
  - encoding the decision tree
  - encoding the additional data which can not be explained by the decision tree

DL with noise

- Code length of decision tree

  CODE: "1 Outlook 0 P 1 Humidity 0 N 0 P 1 Windy 0 N 0 P 1 Temperature 0 N 0 N 0 P"

  LENGTH: 24.585 bits

- Code length of additional data

  LENGTH: 0

  TOTAL LENTH: 24.585 bits

DL without noise

- **Code length of decision tree**
  LENGTH: 13.585 bits

- **Code length of additional data**
  CODE: indicate the position of additional data in possible 54 (3*3*2*3) positions
  LENGTH: 5.75

TOTAL LENTH: 19.335 < 24.585 bits

Learning Decision Tree with MDL

- Use the criterion of minimum DL increase, instead of maximum IG, to choose attributes in the process of generating the tree
- After the tree is generated, prune it at bottom-up mode until DL can not be reduced.

2.4 Learning in Bayesian Inference

- Bayes' rule (also law or theorem or formula)

Class-conditional Probability

Prior Probability

$$P(h|D) = \frac{P(D,h)}{P(D)} = \frac{P(D|h)P(h)}{P(D)}$$

Posterior Probability

h: hypothesis

D: Data

Maximum A Posterior (MAP) Criterion

- Finding the most probable hypothesis
- Determine the MAP hypotheses by using Bayes theorem to calculate the posterior probability of each candidate hypothesis

$$h_{MAP} = \arg\max_h P(h|D) = \arg\max_h \frac{P(D|h)P(h)}{P(D)} = \arg\max_h P(D|h)P(h)$$

An Example: Medical Diagnosis Problem

- Two alternative **hypotheses**
  has a particular form of cancer or not

- The available **data** from a particular laboratory test (+, -)
  P(cancer)=0.008, P(¬cancer)=0.992
  P(+|cancer)=0.98, P(-|cancer)=0.02
  P(+|¬cancer)=0.03, P(-|¬cancer)=0.97

positive result = having cancer?

- $P(+|cancer) \times P(cancer) = 0.98 \times 0.008 = 0.0078$

  $P(+|\neg cancer) \times P(\neg cancer) = 0.03 \times 0.992 = 0.0298$

  According to Bayes theorm :

  $P(cancer|+) = 0.00784 / P(+)$

  $P(\neg cancer|+) = 0.0298 / P(+)$

  $\Longrightarrow$ not cancer

What are learned in Bayesian inference?

- **Class-conditional Probabilities: P(D|h)**

  We should simplify the form of class-conditional probabilities, e.g.,

  - **Naïve Bayesian Classifiers (NBC)**
  - **Bayesian Belief Network (BBN)**
  - **Gaussian Mixture Model (GMM)**
  - **。。。。。。。。**

- **Prior Probabilities: P(h)**

  1. assign the same prior probability to each hypothesis

  2. measure hypothesis frequencies over training data

2.4.1 Naïve Bayesian Classifiers (NBC)

- **Based on the simplifying assumption that the attribute values are conditionally independent given the target value:**

$$P(D|h) = P(d_1, \mathrm{L}\ , d_n|h) = \tilde{O}_i P(d_i|h)$$

  - In discrete problems, learning task is to count the frequency of various data combinations within the training examples.

A NBC Example: Learning to Classify Text Document

- **Target concept**

  "text document that I like or dislike"

- **Training data**

  1000 training documents, 700 labeled 'dislike' , 300 labeled 'like'.

- **Learning process and goal**

  According to text documents for training, we learn Naïve Bayesian Classifiers which can be used to classify new text document.

Example from Tom M. Mitchell, *Machine Learning*

Representation for Text Documents

- Each word position in the document is an attribute whose value is corresponding word.
- E.g. :

  "This is an example document for the naive Bayes classifier. This document contains only one paragraph, or two sentences".

  19 attributes

Learning

- **Prior Probabilities**

  P(dislike)=0.7; P(like)=0.3

- **Class-conditional Probabilities**

  Specific WORD is DEPENDENT of specific POSITION: 2×19×5000 probability terms；

  INDEPENDENT: 2×5000 terms

$$P(w_i|h_j)=(n_j+1)/(n+|Vocubulary|)$$

Classification

$$h_{NB} = \arg \max_{h_j \in \{like, dislike\}} P(h_j) \prod_{i=1}^{19} P(w_i | h_j)$$

$$= \arg \max_{h_j \in \{like, dislike\}} P(h_j) P("this" | h_j) P("is" | h_j) \cdots P("sentences" | h_j)$$

Experimental Results

- **Classifying usenet news articles (Joachims, 1996)**

  Data: 20 newsgroups, each with 1000 articles (2/3 for training, 1/3 for test)

  Accuracy : 89%

- **"usenet articles that I find interesting" (NewsWeeder, Lang, 1995)**

  Allow the user to rate articles, then use them to learn to predict subsequent articles.

  Result: Bring 3~4 times increase.

2.4.2 Bayesian Belief Networks (BBN)

- **Conditional Independence assumption apply to set of attributes.**
- **X is conditionally independent of Y given Z**

the probability distribution governing X is independent of the value of Y given a value of Z

$$P(x|y,z)=P(x|z)$$

extend to sets of variables :

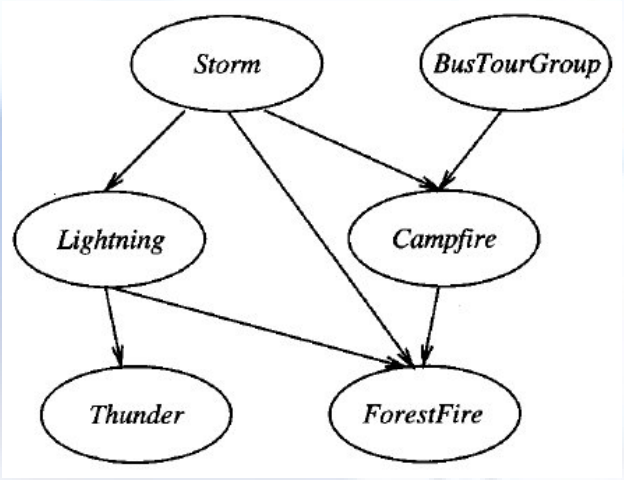
$$P(x_1,\mathrm{L}\ ,x_l|y_1,\mathrm{L}\ ,y_m,z_1,\mathrm{L}\ ,z_n)=P(x_1,\mathrm{L}\ ,x_l|z_1,\mathrm{L}\ ,z_n)$$

Bayesian Belief Networks （BBN）

- **Joint probability can be computed by**

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid Parents(x_i))$$

which can be represented as a graph, E. g,



Conditional probability table (CPT)

	S,B	S,¬B	¬S,B	¬S,¬B
C	0.4	0.1	0.8	0.2
¬C	0.6	0.9	0.2	0.8

Campfire

also represent casual knowledge

Tom M. Mitchell, *Machine Learning*

Compactness

- A CPT for Boolean X_i with k Boolean parents has 2^k rows for the combinations of parent values. Each row requires one number p for X_i =true

- If each variable has no more than k parents, the complete network requires numbers $O(n \times 2^k)$, i.e., grows linearly with n, vs. $O(2^n)$ for the full joint distribution

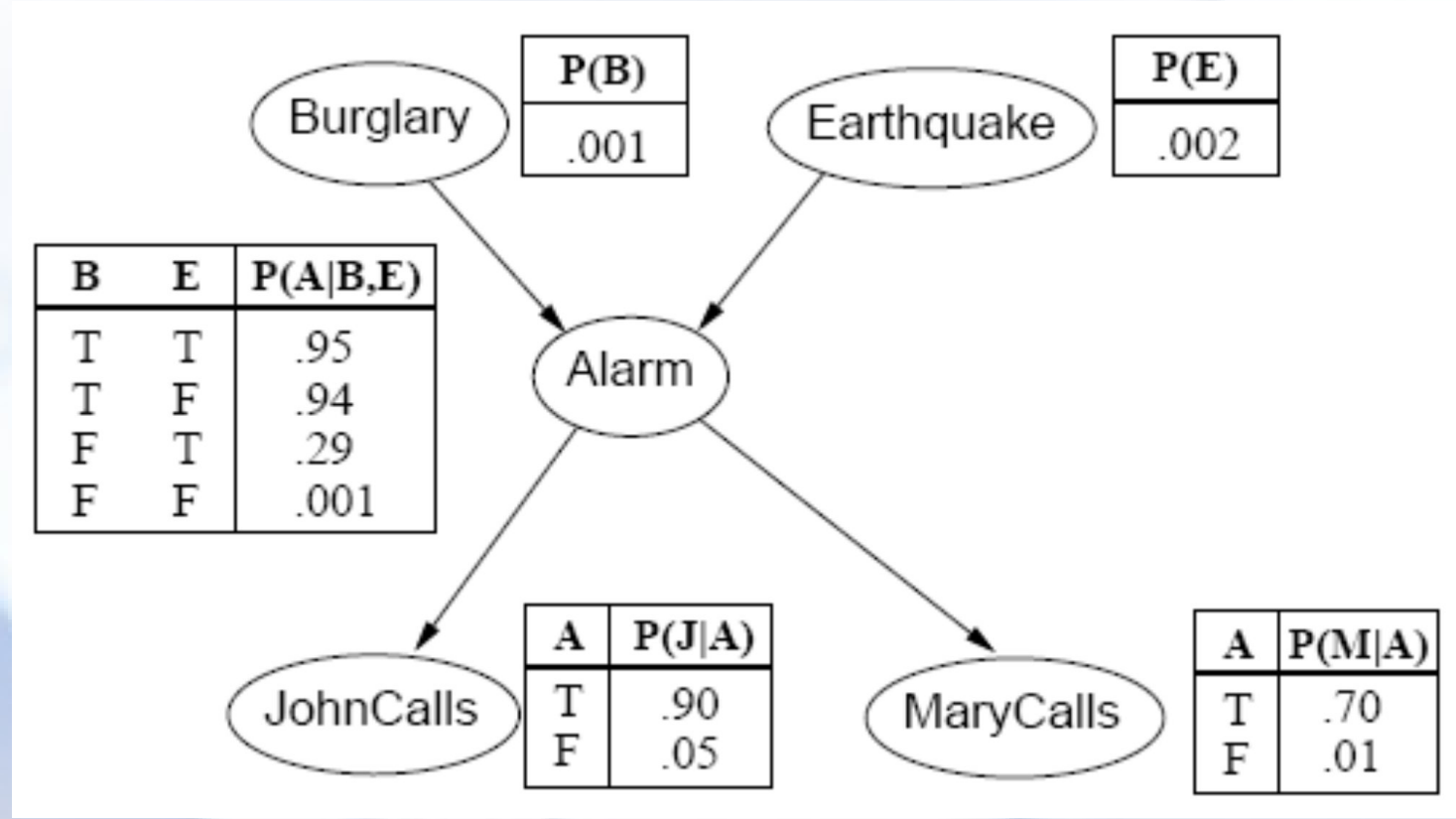
An example of BBN

- I'm at work, neighbor John calls to say my alarm is ringing, but neighbor Mary doesn't call. Sometimes it's set off by minor earthquakes. Is there a burglar?

- **Variables**: Burglar, Earthquake, Alarm, JohnCalls, MaryCalls

S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*

Example contd.



Example contd.

- Network topology reflects “causal" knowledge:
  - A burglar can set the alarm off
  - earthquake can set the alarm off
  - The alarm can cause Mary to call
  - The alarm can cause John to call

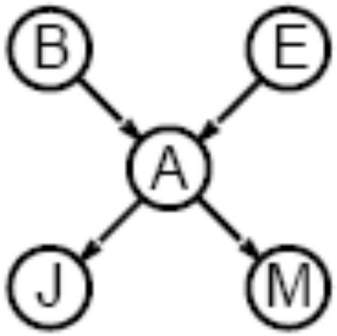
Example contd.

- **The full joint distribution as the product of the local conditional distributions:**

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i | parents(X_i))$$

e.g., $P(j \wedge m \wedge a \wedge \neg b \wedge \neg e)$

$$\begin{aligned} &= P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e) \\ &= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998 \\ &\approx 0.00063 \end{aligned}$$



Inference in Bayesian Networks

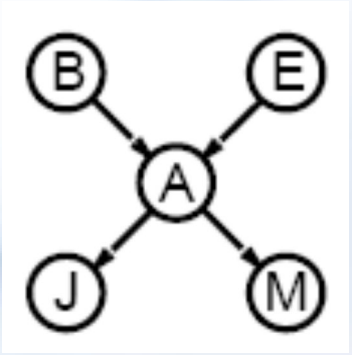
- Exact inference by enumeration
- Exact inference by variable elimination
- Approximate inference by stochastic simulation
- Approximate inference by Markov chain Monte Carlo

Exact Inference by Enumeration

- **Any conditional probability can be computed by summing terms from the full joint distribution.**

- **E.g,** $P(x|y) = \alpha P(x,y) = \alpha \sum_z P(x,y,z)$

$$\begin{aligned}
&\mathbf{P}(B|j,m) \\
&= \mathbf{P}(B,j,m)/P(j,m) \\
&= \alpha \mathbf{P}(B,j,m) \\
&= \alpha \textstyle\sum_e \sum_a \mathbf{P}(B,e,a,j,m)
\end{aligned}$$



Exact Inference by Enumeration

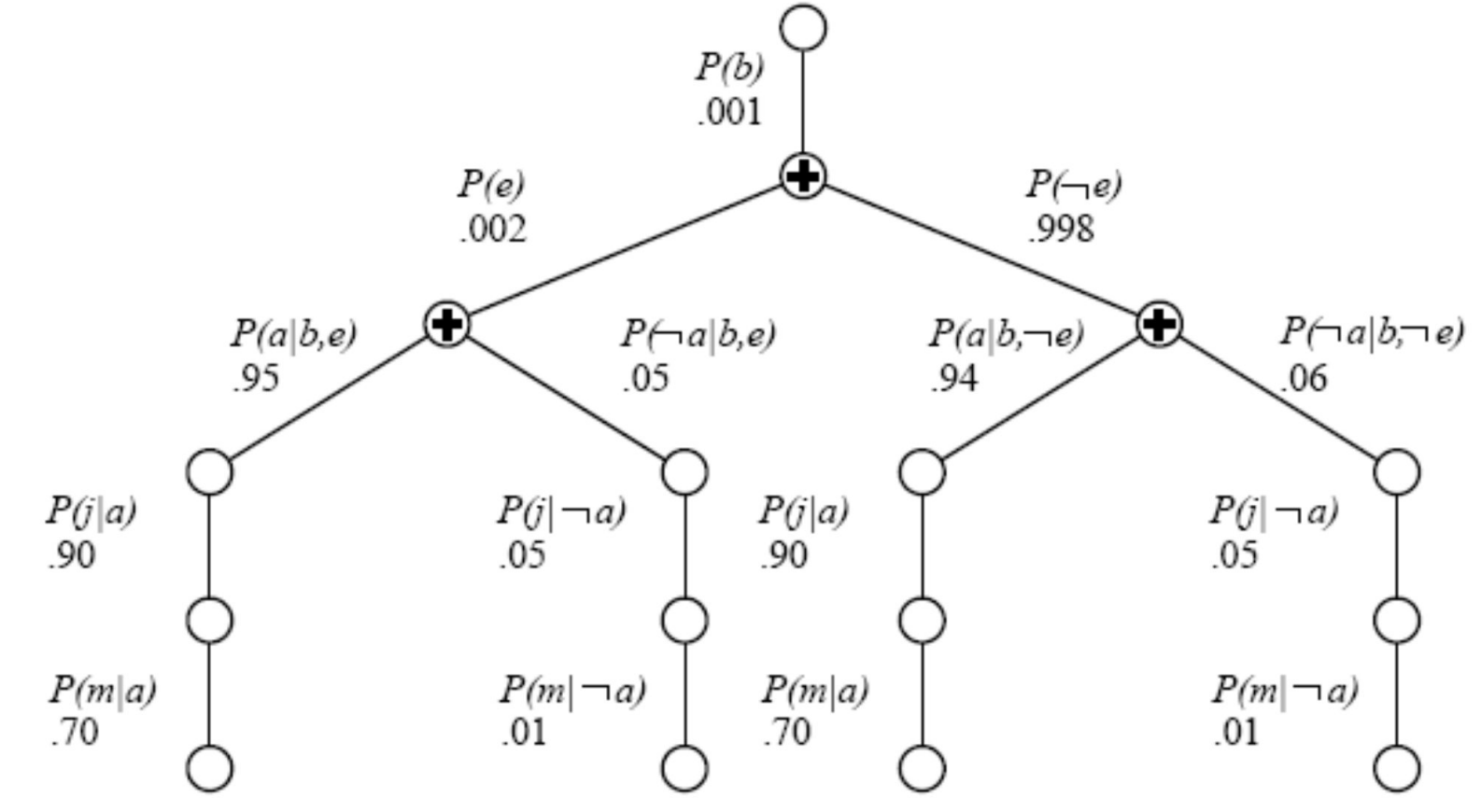
- **Rewrite full joint entries using product of CPT entries:**

$$\begin{aligned}&\mathbf{P}(B|j,m)\\&=\alpha\sum_e\sum_a\mathbf{P}(B)P(e)\mathbf{P}(a|B,e)P(j|a)P(m|a)\\&=\alpha\mathbf{P}(B)\sum_e P(e)\sum_a\mathbf{P}(a|B,e)P(j|a)P(m|a)\end{aligned}$$

$$\mathbf{P}(B|j,m)=\alpha\langle 0.00059224,0.0014919\rangle$$

$$\approx\langle 0.284,0.716\rangle$$

Exact Inference by Enumeration



Inefficient valuation ⟶ Variable elimination

Learning Bayesian Belief Networks

- **Learning the conditional probability tables**

  1. the variables are fully observable

     simply measure over training data

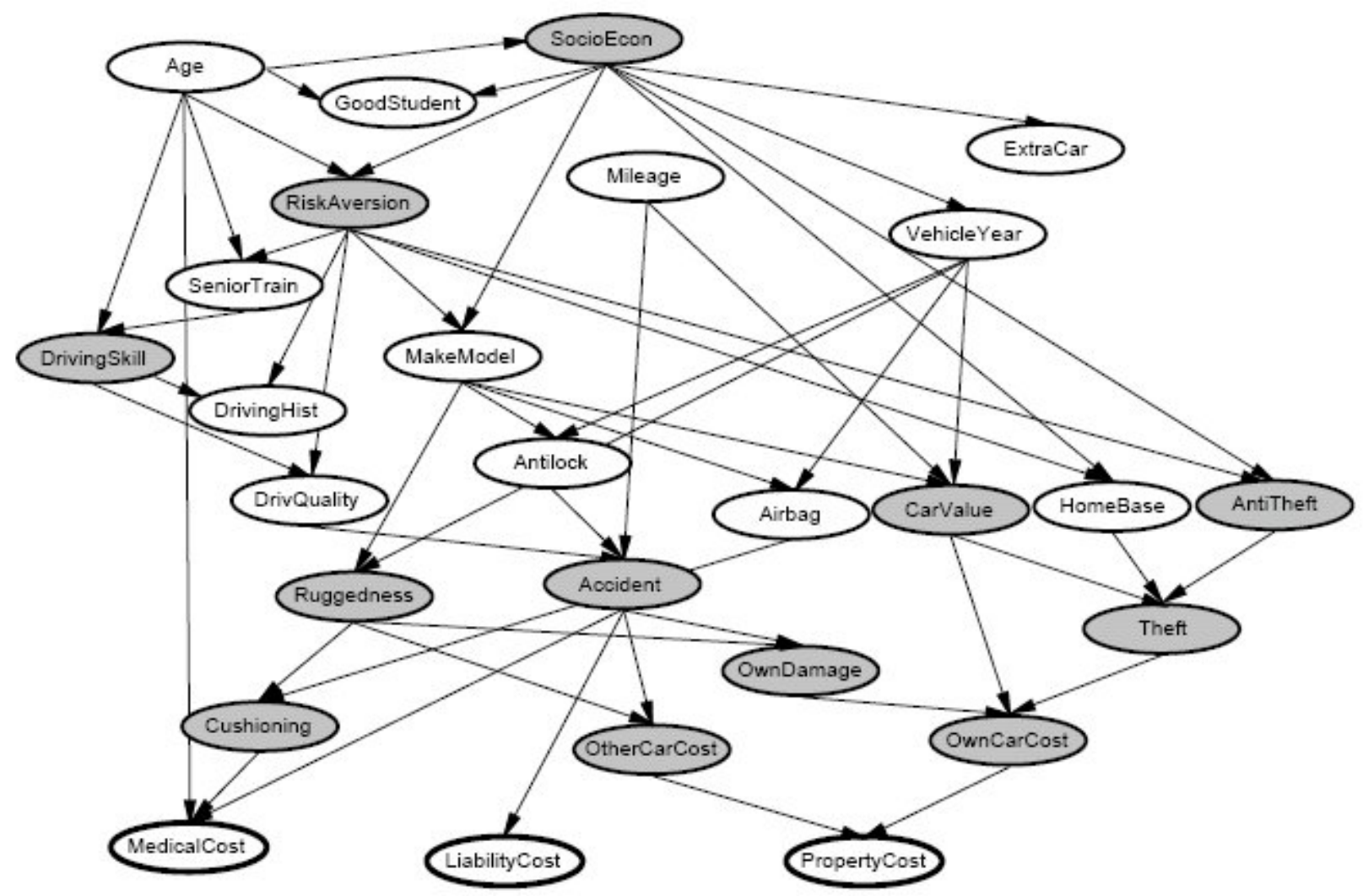
  2. only some of the variables are observed

     similar with Multi-layer Perceptron

- **Learning the network structure**

  Difficult

An Example of BBN Learning: Insurance Risk Estimation



Gradient Ascent Training

- **Objective (Maximum Likelihood Estimation, MLE)**
  maxPw(D), maxlnPw(D) ☾ maxPw(D)
- **Optimization method: Gradient Ascent**

$$\frac{\partial \ln P_w(D)}{\partial w_{ijk}} = \frac{\partial}{\partial w_{ijk}} \ln \prod_{d \in D} P_w(d)$$

$$w_{ijk} = w_{ijk} + \eta \frac{\partial \ln P_w(D)}{\partial w_{ijk}}$$

$$= \sum_{d \in D} \frac{\partial \ln P_w(d)}{\partial w_{ijk}} = \sum_{d \in D} \frac{1}{P_w(d)} \frac{\partial P_w(d)}{\partial w_{ijk}}$$

Gradient Ascent Training

$$\frac{\partial P_w(d)}{\partial w_{ijk}} = \frac{\partial}{\partial w_{ijk}} \sum_{j',k'} P_w\left(d \middle| x_{ij'}, u_{ik'}\right) P_w\left(x_{ij'}, u_{ik'}\right)$$

$$= \frac{\partial}{\partial w_{ijk}} \sum_{j',k'} P_w\left(d \middle| x_{ij'}, u_{ik'}\right) P_w\left(x_{ij'} \middle| u_{ik'}\right) P_w\left(u_{ik'}\right) = P_w\left(d \middle| x_{ij}, u_{ik}\right) P_w\left(u_{ik}\right)$$

$$\frac{\partial \ln P_w(D)}{\partial w_{ijk}} = \sum_{d \in D} \frac{1}{P_w(d)} P_w\left(d \middle| x_{ij}, u_{ik}\right) P_w\left(u_{ik}\right)$$

$$= \sum_{d \in D} \frac{1}{P_w(d)} \frac{P_w\left(x_{ij}, u_{ik} \middle| d\right) P_w(d) P_w\left(u_{ik}\right)}{P_w\left(x_{ij}, u_{ik}\right)} = \sum_{d \in D} \frac{P_w\left(x_{ij}, u_{ik} \middle| d\right) P_w\left(u_{ik}\right)}{P_w\left(x_{ij}, u_{ik}\right)}$$

$$= \sum_{d \in D} \frac{P_w\left(x_{ij}, u_{ik} \middle| d\right)}{P_w\left(x_{ij} \middle| u_{ik}\right)} = \sum_{d \in D} \frac{P_w\left(x_{ij}, u_{ik} \middle| d\right)}{w_{ijk}}$$

Part III: Unsupervised Learning: Clustering

3.1 What is Clustering?

- Classical unsupervised learning
  only input data, but expected output is unknown
- The process of dividing the data set into clusters according to the similarity between data.
- Discovering groups and Identifying interesting distributions in the underlying data.

Three key problems

- How to measure the similarity between data or clusters?
- How to divide the data set?
- How to deal with the large dataset and high dimensionality? (scalability)

Data Similarity Measure

- Distance between two ordered data should satisfy the following axiom:

  1. $d(i,j) \geq 0$, 2. $d(i,i) = 0$, 3. $d(i,j) = d(j,i)$

  4. $d(i,j) \leq d(i,k) + d(k,j)$

- Widely used distance include Euclidean, City block, Chebyishev, Minkowski, and etc.
- Statistical measure: Represent clusters by using statistical models such as Gaussian. Then Class-conditional probability density can be used to measure the degree of a data belonging to the cluster.

3.2 Partition clustering

- Partition data set into *k* subsets (clusters) under defined criterion.
- Search the optimal partition in the set of all possible partitions
  - Blind search
  - Heuristic search

Heuristic search based partition clustering

- ***k*-means algorithm (MacQueen'67)**

  Represent each cluster using its mean vector.

- ***k*-medoids algorithm**

  Represent each cluster using one of data in the set, called medoid or representative data.

3.2.1 *k*-means algorithm

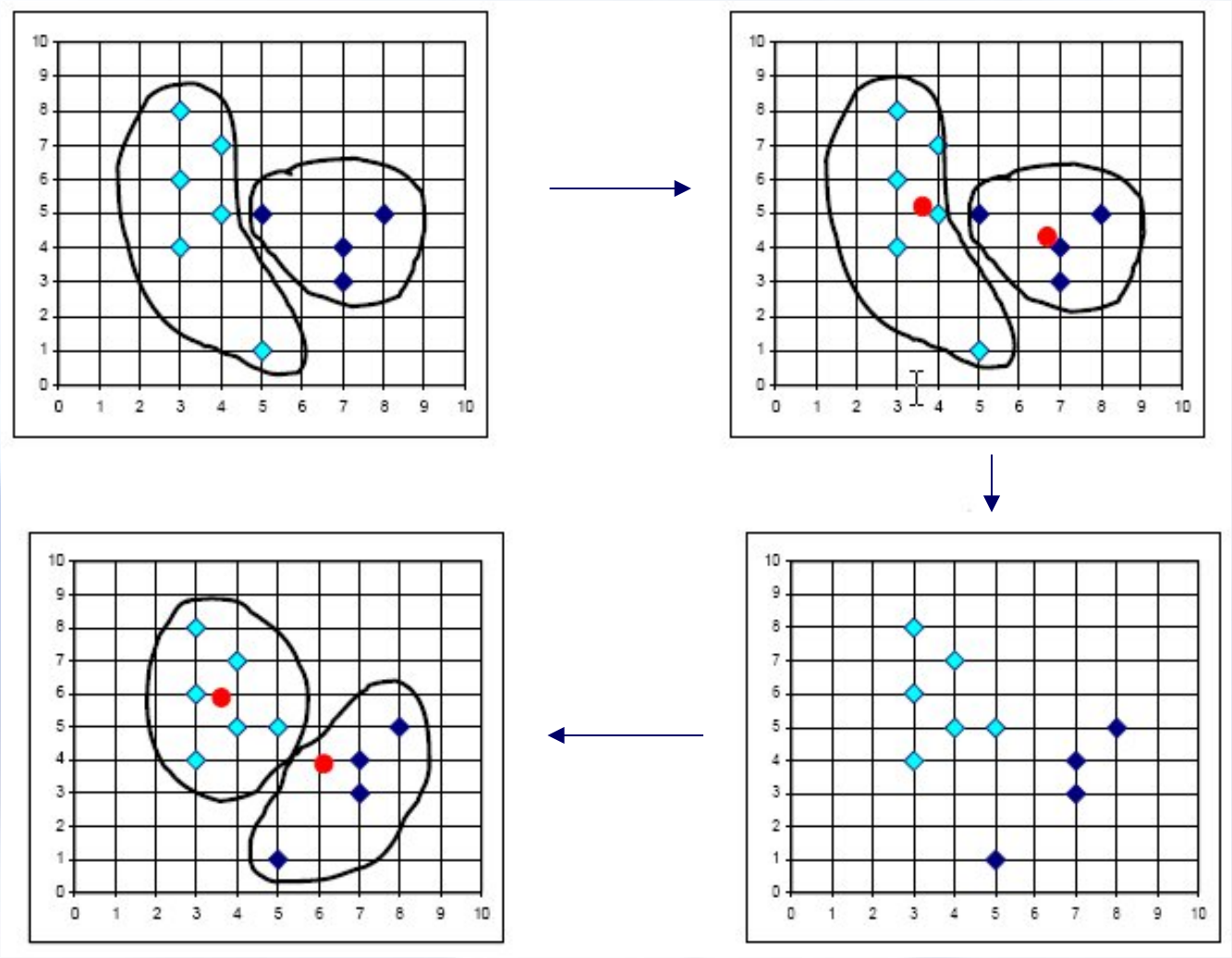
Step1.Randomly select *k* data from the set as the initial mean vectors of clusters.

Step2.Other data are partitioned into the closest cluster measured by its distance to corresponding mean vector.

Step3.Update the mean vector of new generated clusters.

Step4.Repeat Step2-3, until clustering is stable.

An example of *K*-means clustering



From J. Han and M. Kamber, "***Data Mining: Concepts and Techniques***".

Objective function

$$J(X,v)=\sum_{i=1}^{n}\sum_{j=1}^{k}(u_{ik})^{m}\,r\left(d(x_i,v_j)\right)$$

mean

Distance measure

Membership matrix : $\sum_{i=1}^{k} u_{ik}=1$

k-means

$$r\left(d(x_i,v_j)\right)=\|x_i - v_j\|^2 \qquad m=1 \qquad u_{ik}\in\{0,1\}$$

m could be 1,2,3… $u_{ik}\in[0,1]$

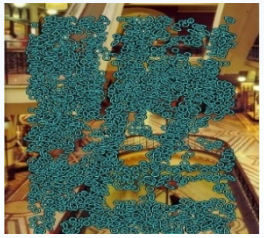
Fuzzy *k*-means, FCM

Features of *k*-means algorithm

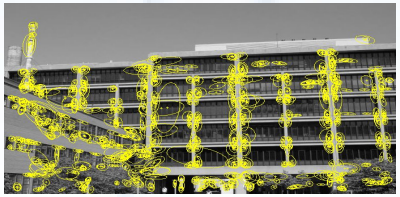
- Time Complexity : O(*tkn*)
- Local Optimality
- Need to preset the number of clusters (*k*)
- Sensitive to noises and initial means

K-means Applications: bag-of-visual-words

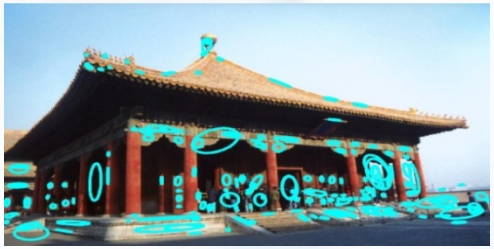
- **Bag-of-visual-words: A popular image representation based on local features**



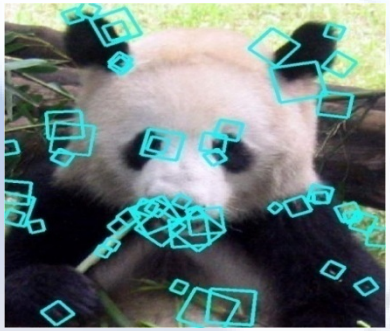
Salient regions
[Kadir 01]



Harris-Affine
[Mikolajczyk 04]



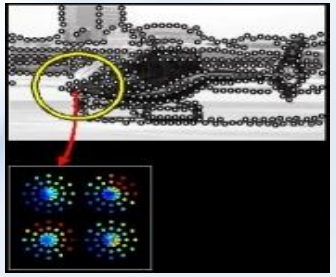
Maximally Stable Extremal Regions
[Matas 02]



SIFT [Lowe 99]



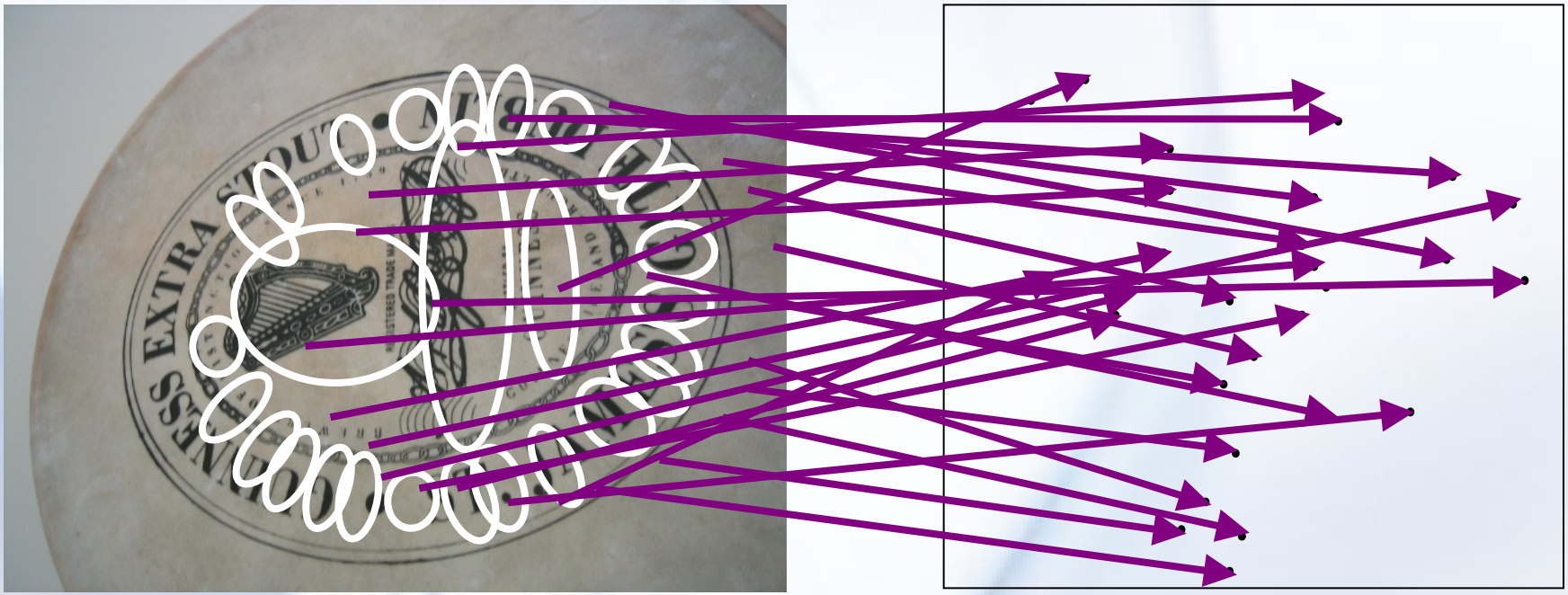
Shape context
[Belongie 02]



Geometric Blur
[Berg 05]

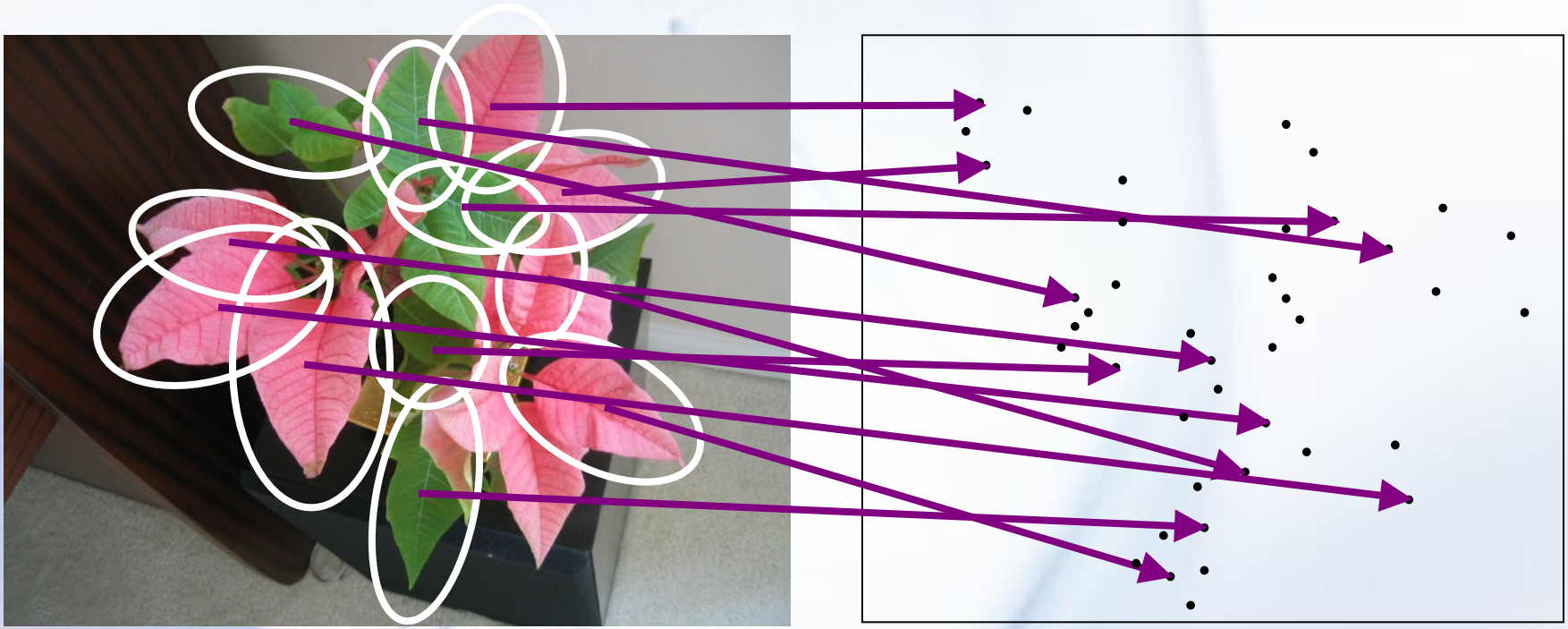
Build Visual Words

- **Step1. Extract local features from many images :**



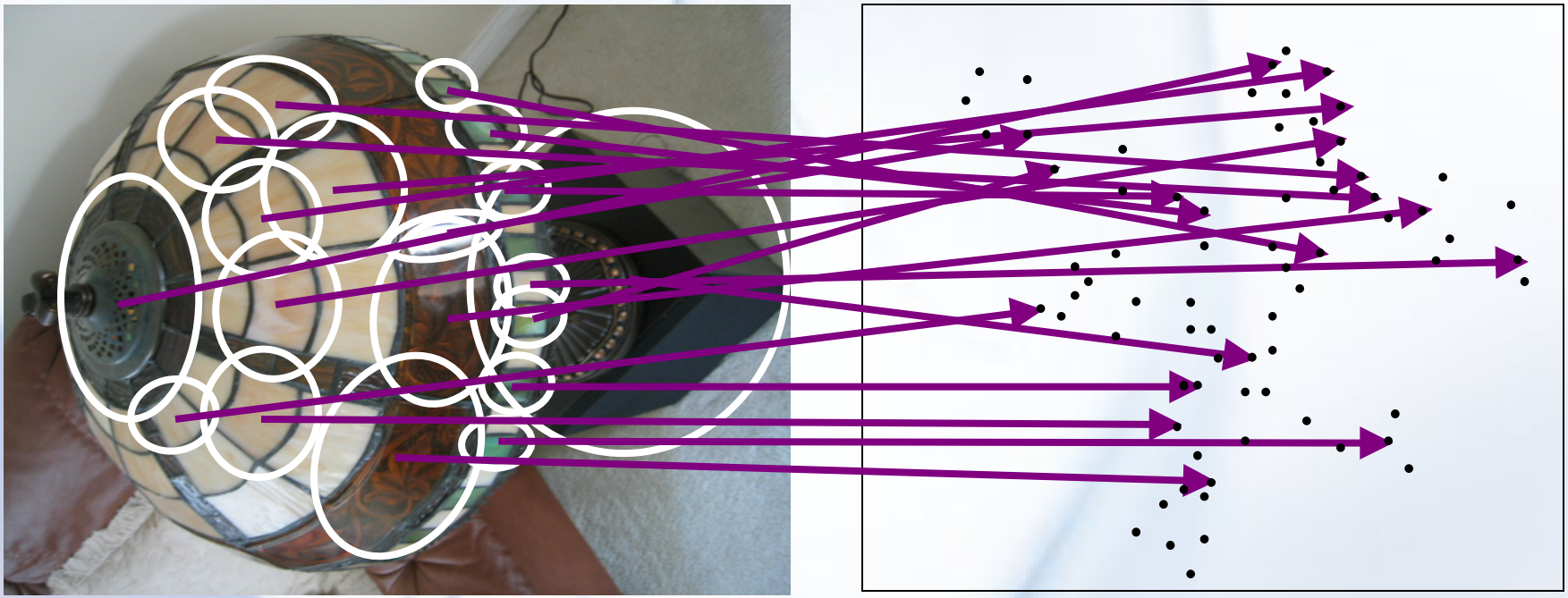
Build Visual Words

- Step1. Extract local features from many images :



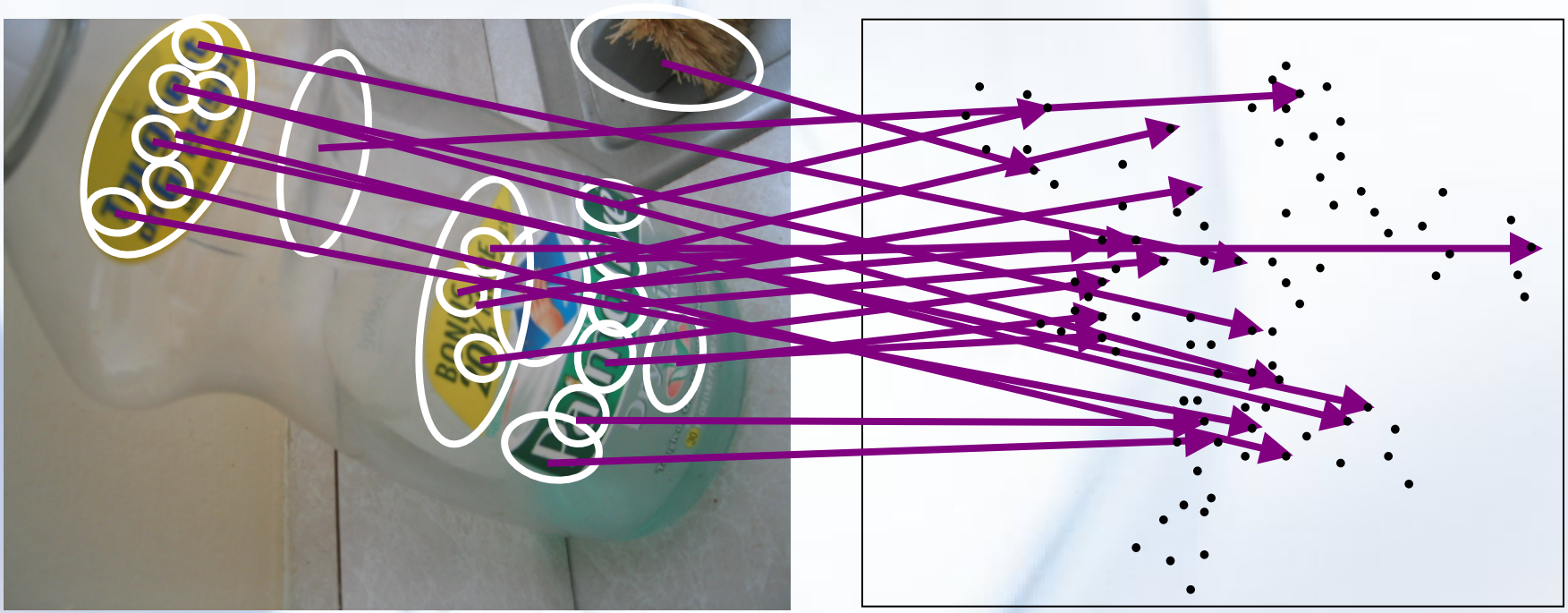
Build Visual Words

- Step1. Extract local features from many images :



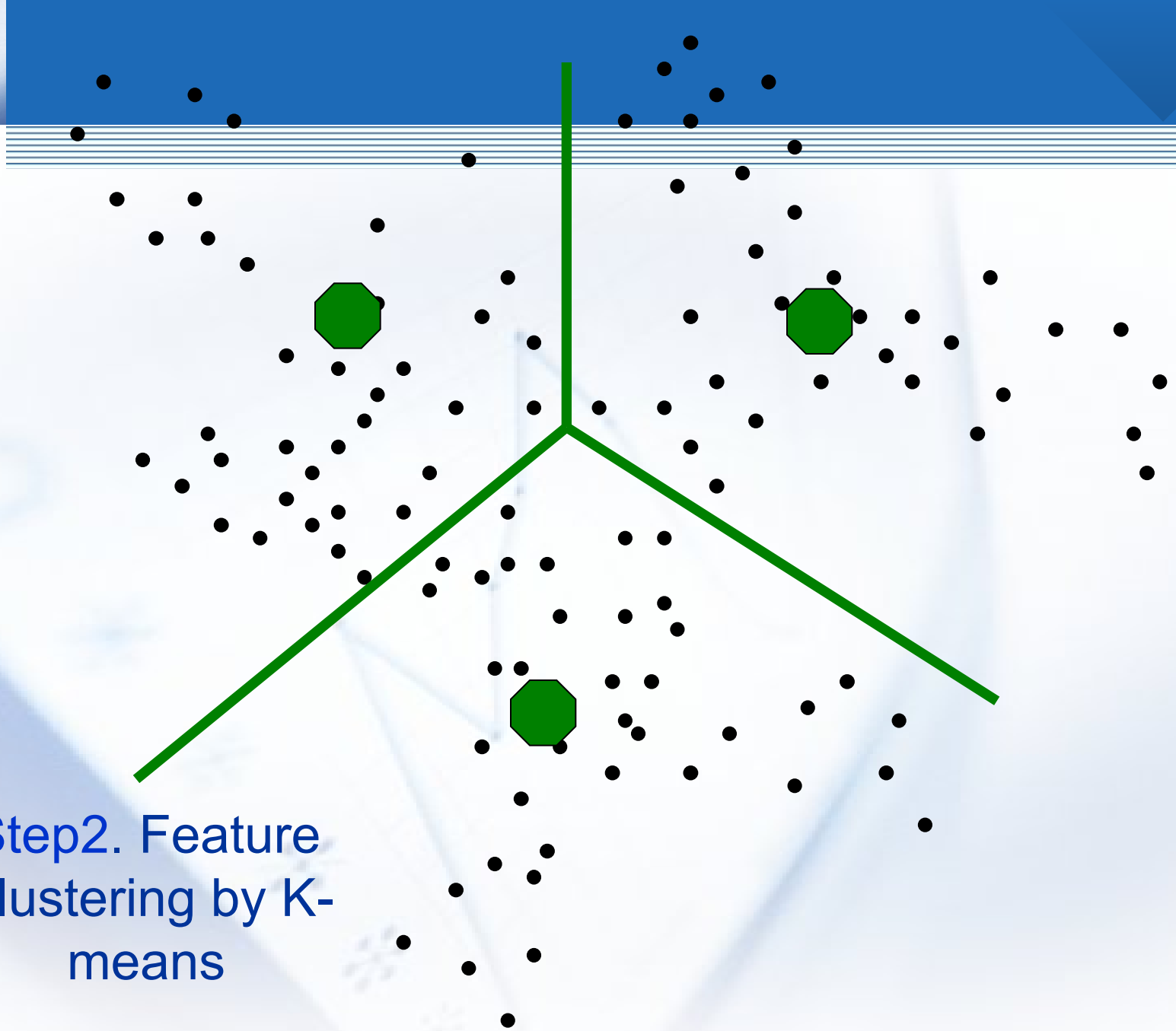
Build Visual Words

- Step1. Extract local features from many images :



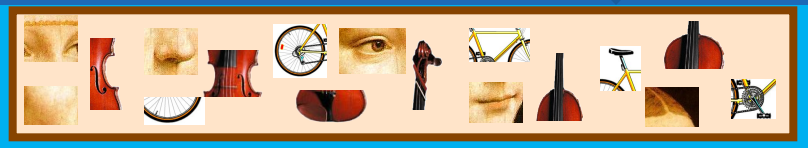


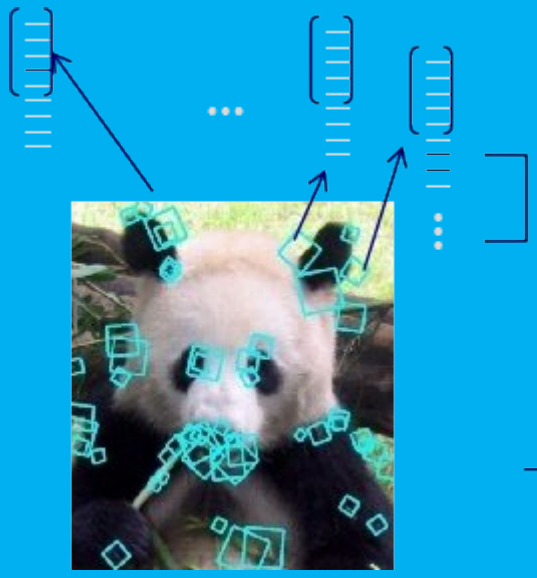
Step2. Feature Clustering by K-means



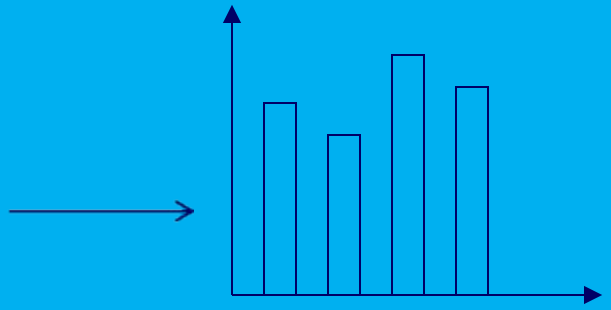
Step2. Feature Clustering by K-means

Step3. Represent images by visual words

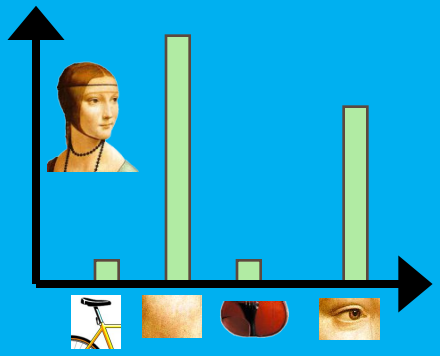


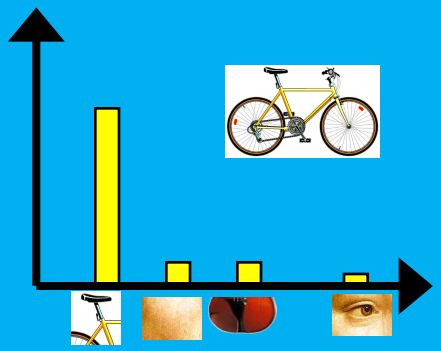


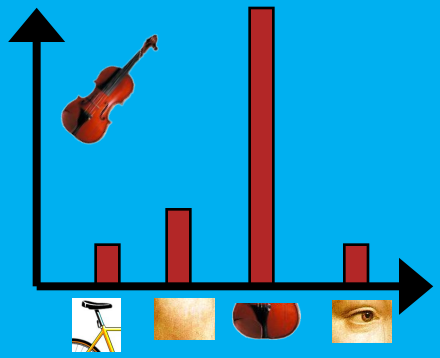
Local Feature and their Descriptors



Histogram = Image Representation







3.2.2 *k*-medoids algorithm

- Also called PAM (Partitioning Around Medoids)

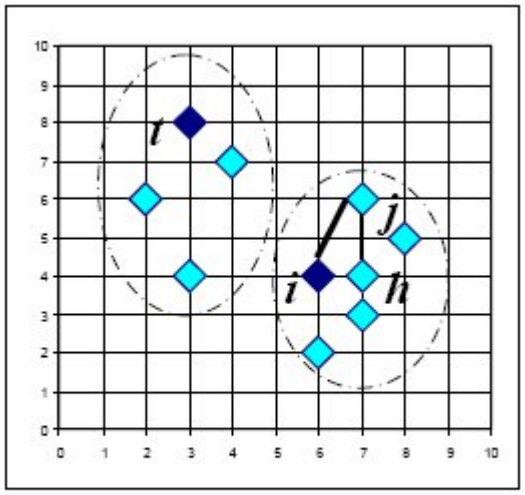
  Use the medoid (representative data) , instead of the mean, to represent each cluster.

- Idea

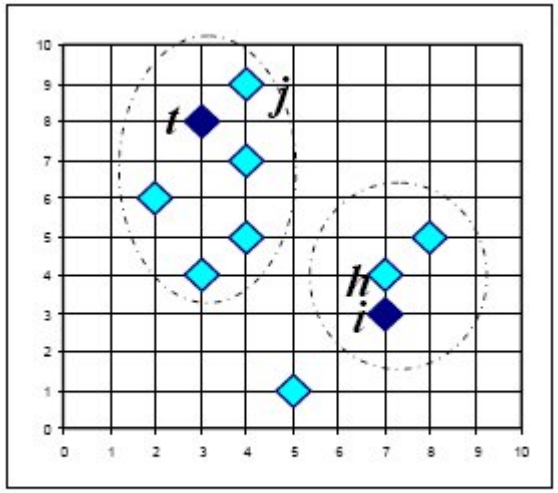
  1.Data is partitioned according to its distance to medoids.

  2.Improve clustering quality through iteratively substituting representative data by other data.

Substitution Cost

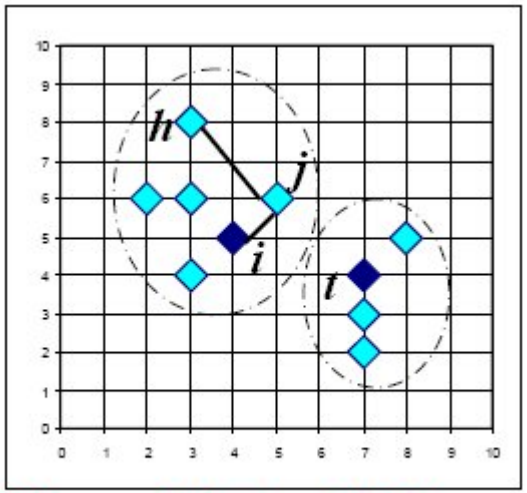


$C_{jih} = d(j, h) - d(j, i)$

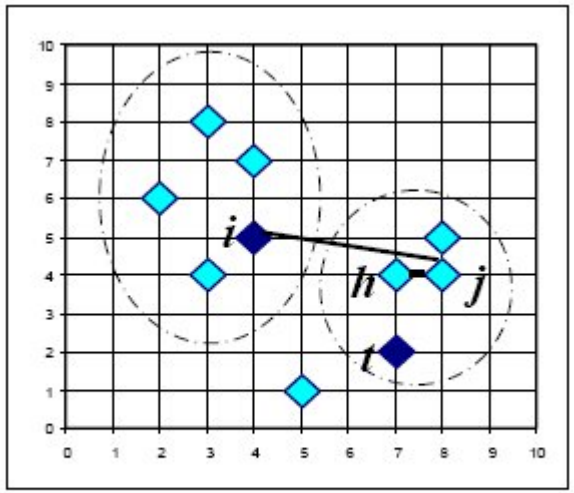


$C_{jih} = 0$

Substitution cost of each data



$C_{jih} = d(j, t) - d(j, i)$



$C_{jih} = d(j, h) - d(j, t)$

Total substitution cost for the data set:

$$C_{ih} = \sum_j C_{jih}$$

PAM algorithm

Step1. Randomly select *k* mediods

Step2. Compute total substitution cost for each pair of mediod *h* and other data *i*, let it be TC_{ih}

Step3. If the minimum $TC_{ih}<0$, then *i* is substituted by *h*

Step4.Repeat Step2-3,until the minimum $TC_{ih} \geqq 0$.

Step5.Partition other data into the closet cluster measured by its distance to mediods.

Improvement of PAM

- Improve the efficiency, especially for large data set.
- Improved algorithms

  CLARA(Clustering LARge Applications),1990

  Find optimal mediods on the sampled subset.

  CLARANS (Random CLARA), 1994

  Improve CLARA by repetitive sample

CLARA

Algorithm CLARA

1. For $i = 1$ to 5, repeat the following steps:
2. Draw a sample of $40+2k$ objects randomly from the entire data set [2], and call Algorithm PAM to find k medoids of the sample.
3. For each object O_j in the entire data set, determine which of the k medoids is the most similar to O_j.
4. Calculate the average dissimilarity of the clustering obtained in the previous step. If this value is less than the current minimum, use this value as the current minimum, and retain the k medoids found in Step (2) as the best set of medoids obtained so far.
5. Return to Step (1) to start the next iteration. □

From: Ng and Han, CLARANS

CLARANS

- **Clustering Large Applications based upon RANdomized Search**
- **The problem space: graph of clustering**
  - A vertex is k from n numbers, i.e., a solution for k-mediods clustering, two vertex are connected if their sets differ by only one mediods
  - PAM search the whole graph
  - CLARA search some random sub-graphs
- **CLARANS Search: hill-climb**
  - Randomly select a node (ie., k medoids)
  - Consider a set of randomly chosen neighbor nodes as candidate of new medoids
  - Moves to the neighbor node if the neighbor is a better choice for medoids. Otherwise, a local optima is discovered.
  - The entire process is repeated multiple time to find better local optima.

CLARANS contd.

Algorithm CLARANS

1. Input parameters *numlocal* and *maxneighbor*. Initialize i to 1, and *mincost* to a large number.
2. Set *current* to an arbitrary node in $G_{n,k}$.
3. Set j to 1.
4. Consider a random neighbor S of *current*, and based on Equation (5), calculate the cost differential of the two nodes.
5. If S has a lower cost, set *current* to S, and go to Step (3).
6. Otherwise, increment j by 1. If $j \leq maxneighbor$, go to Step (4).
7. Otherwise, when $j > maxneighbor$, compare the cost of *current* with *mincost*. If the former is less than *mincost*, set *mincost* to the cost of *current*, and set *bestnode* to *current*.
8. Increment i by 1. If $i > numlocal$, output *bestnode* and halt. Otherwise, go to Step (2). □

substitution cost

From: Ng and Han, CLARANS

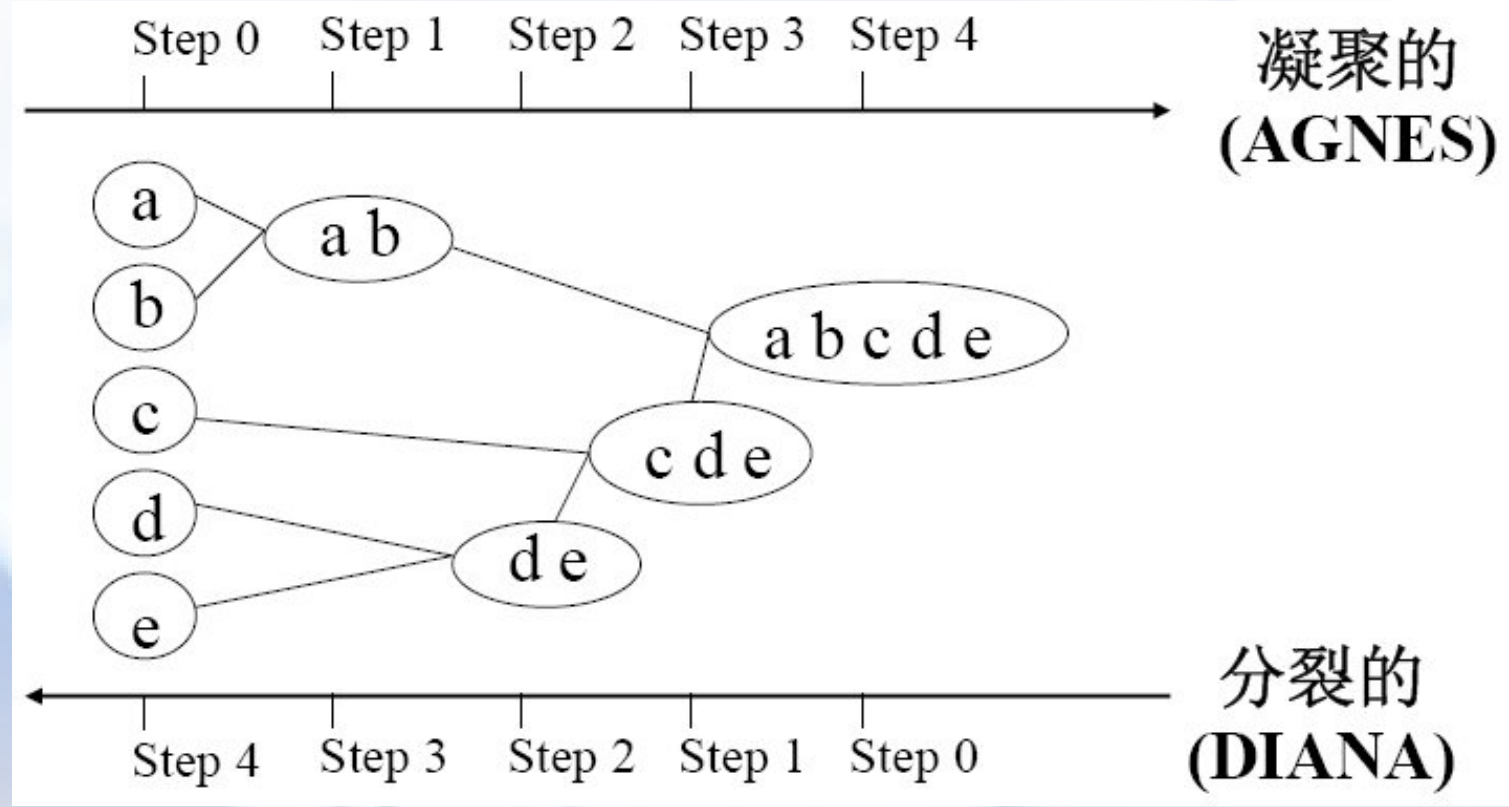
3.3 Hierarchical clustering

- **Similarity measure between clusters is important.**
- **Bottom-up Agglomerative Nesting**

  Firstly take each data as a cluster, then combine them gradually
- **Top-down Divisive Analysis**

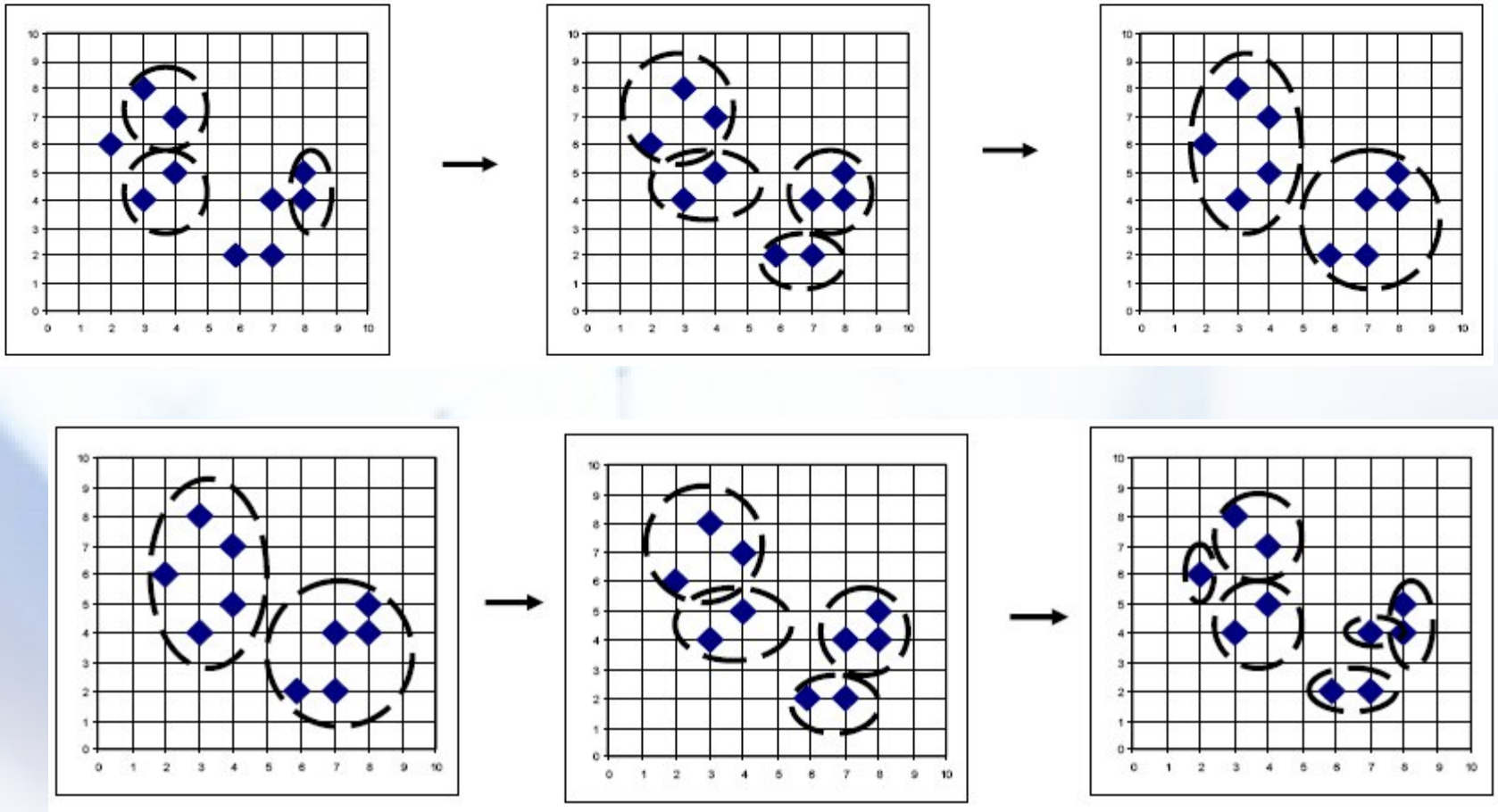
  Firstly take the whole set as a cluster, then split them gradually

Agglomerative Nesting and Divisive Analysis



AGNES and DIANA

AGNES(AGglomerative NESting)



DIANA (DIvisive ANAlysis)

Cluster distance and Hierarchical clustering

- Single-Linkage algorithm
  distance = minimum distance between data
- Complete-Linkage algorithm
  distance = maximum distance between data
- Average-Linkage algorithm
  distance = the mean distance between data
- Centroid-Linkage algorithm, Median-Linkage algorithm, Ward algorithm .....

Problems and Solutions

- "One false step will make a great difference".
- Unsatisfactory efficiency
- Improved algorithms

  BIRCH (1996): treat the process of hierarchal clustering as a tree.

  CURE (1998): use a set of well-scattered points to represent each cluster

  CHAMELEON (1999): use dynamic model

(1) BIRCH algorithm

- **Balanced Iterative Reducing and Clustering using Hierarchies**
- **To calculate centroid, radius, diameter and corresponding distance between clusters, not all points are needed**
- **3 values are stored to represent the cluster, called clustering feature (CF):**

$$\mathbf{CF} = (N, \vec{LS}, SS)$$

- **N – number of points in a cluster**
- **LS – linear sum of points in a cluster**
- **SS – square sum of points in a cluster**

CF Example

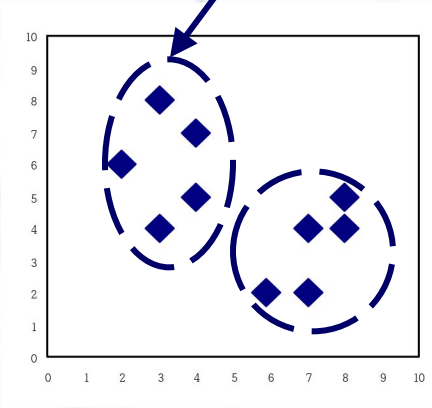
Clustering Feature: ***CF = (N, LS, SS)***

***N*: Number of data points**

LS: $\sum_{i=1}^{N} X_i$

SS: $\sum_{i=1}^{N} X_i^2$

CF = (5, (16,30),(54,190))



(3,4)
(2,6)
(4,5)
(4,7)
(3,8)

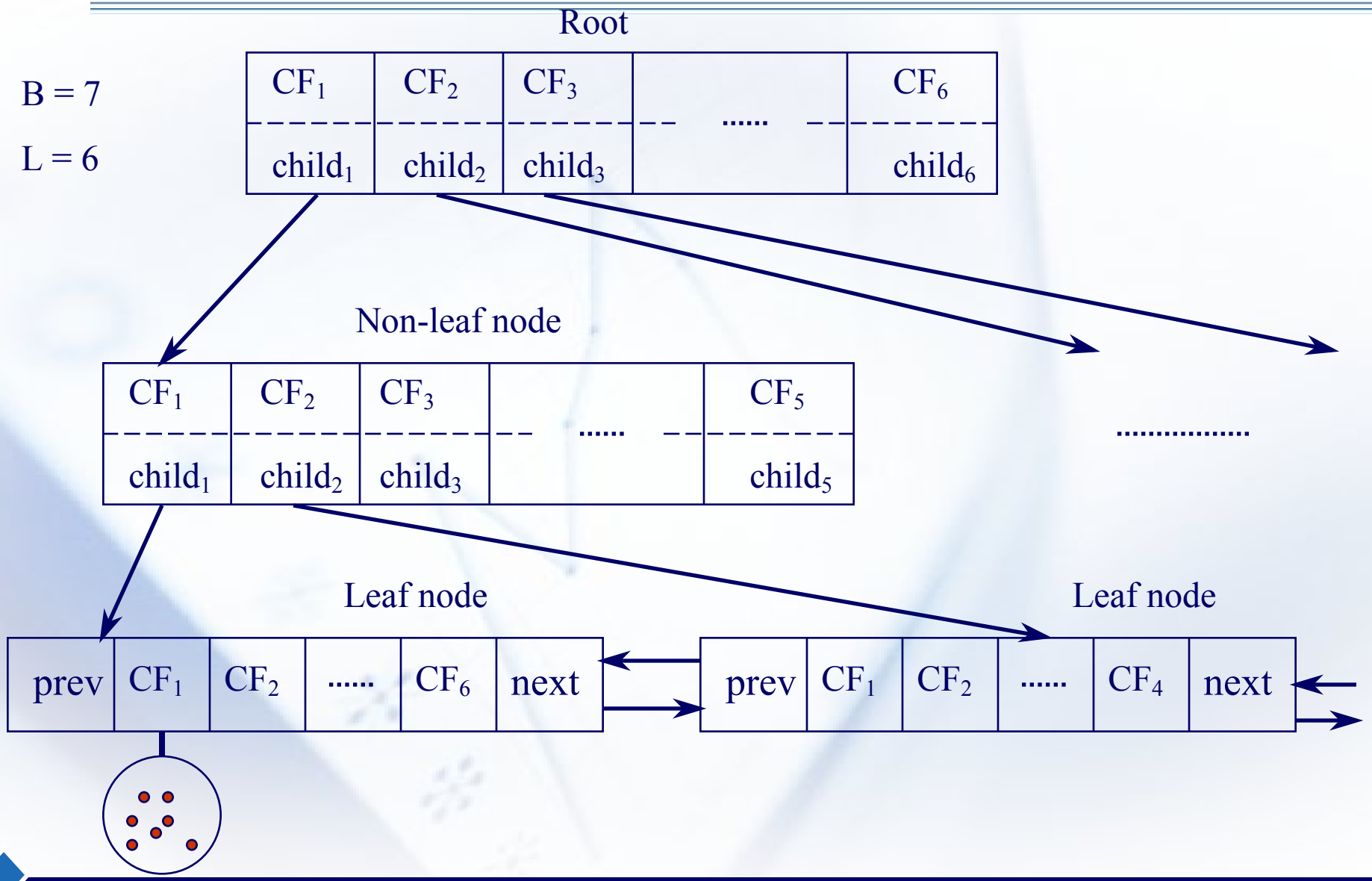
- **CF are additive:**

$$\mathbf{CF_1} + \mathbf{CF_2} = (N_1 + N_2, L\vec{S}_1 + L\vec{S}_2, SS_1 + SS_2)$$

CF Tree

- **CF tree is similar to B^+-tree, R-tree**
- **Parameters**
  - B, L – branching factor
  - T – threshold
- **Leaf node – contains at most L CF entries, each CF should follows D<T or R<T**
- **Non-leaf node – contains at most B CF entries of its child**
- **Each node should fit into 1 page**

CF Tree Example



BIRCH algorithm

- **Incrementally construct a CF (Clustering Feature) tree**
  - Parameters: max diameter (L), max children (B)
  - Phase 1: scan DB to build an initial in-memory CF tree (each node: #points, sum, sum of squares)
  - Phase 2: use an arbitrary clustering algorithm to cluster the leaf nodes of the CF-tree
- ***Scales linearly*: finds a good clustering with a single scan and improves the quality with a few additional scans**
- ***Weaknesses:* handles only numeric data, sensitive to order of data records.**

Building CF Tree

- **Insert a point to the tree**
  - Find the path (based on the distance between CF of children in a non-leaf node)
  - Modify the leaf
    - Find closest leaf node entry (based on the distance of CF in leaf node)
    - Check if it can “absorb” the new data point
  - Modify the path to the leaf
  - Splitting – if leaf node is full, split into two leaf node, add one more entry in parent

Global Clustering

- Consider CF entries in leaf nodes only
- Use centroid as the representative of a cluster
- Perform traditional clustering (e.g. agglomerative or K-mean…)
- Cluster CF instead of data points

(2) CURE algorithm

- **Clustering Using REpresentatives**

  A kind of agglomerative nesting algorithm. The main features include: 1) represent a cluster using several data; 2) The representative points are shrunk toward the mean gradually; 3) apply the techniques of random sampling and partitioning for large data sets

Guha, Rastogi, and Shim, 1998

Core: CURE hierarchal clustering

```
procedure cluster(S, k)
begin
1.  T := build_kd_tree(S)
2.  Q := build_heap(S)
3.  while size(Q) > k do {
4.      u := extract_min(Q)
5.      v := u.closest
6.      delete(Q, v)
7.      w := merge(u, v)
8.      delete_rep(T, u); delete_rep(T, v); insert_rep(T, w)
9.      w.closest := x /* x is an arbitrary cluster in Q */
10.     for each x ∈ Q do {
11.         if dist(w, x) < dist(w, w.closest)
12.             w.closest := x
13.         if x.closest is either u or v {
14.             if dist(x, x.closest) < dist(x, w)
15.                 x.closest := closest_cluster(T, x, dist(x, w))
16.             else
17.                 x.closest := w
18.             relocate(Q, x)
19.         }
20.         else if dist(x, x.closest) > dist(x, w) {
21.             x.closest := w
22.             relocate(Q, x)
23.         }
24.     }
25.     insert(Q, w)
26. }
end
```

Until k clusters

Determine the clusters for merge

Recompute the closest cluster

Perform merge

```
procedure merge(u, v)
begin
1.  w := u ∪ v
2.  w.mean := (|u|u.mean+|v|v.mean) / (|u|+|v|)
3.  tmpSet := ∅
4.  for i := 1 to c do {
5.      maxDist := 0
6.      foreach point p in cluster w do {
7.          if i = 1
8.              minDist := dist(p, w.mean)
9.          else
10.             minDist := min{dist(p, q) : q ∈ tmpSet}
11.         if (minDist ≥ maxDist){
12.             maxDist := minDist
13.             maxPoint := p
14.         }
15.     }
16.     tmpSet := tmpSet ∪ {maxPoint}
17. }
18. foreach point p in tmpSet do
19.     w.rep := w.rep ∪ {p + α*(w.mean-p) }
20. return w
end
```

Select representative points

Point shrunk

CURE algorithm

Step1. Sample the data set，get the sample set

Step2. Partition the sample set into *p* parts, each of which include *n*/*p* samples.

Step3. Perform CURE hierarchal clustering on each part, the cluster number is set to be *n*/*pq*

Step4. Delete singular points (very small cluster)

Step5. Perform CURE hierarchal clustering on generated clusters.

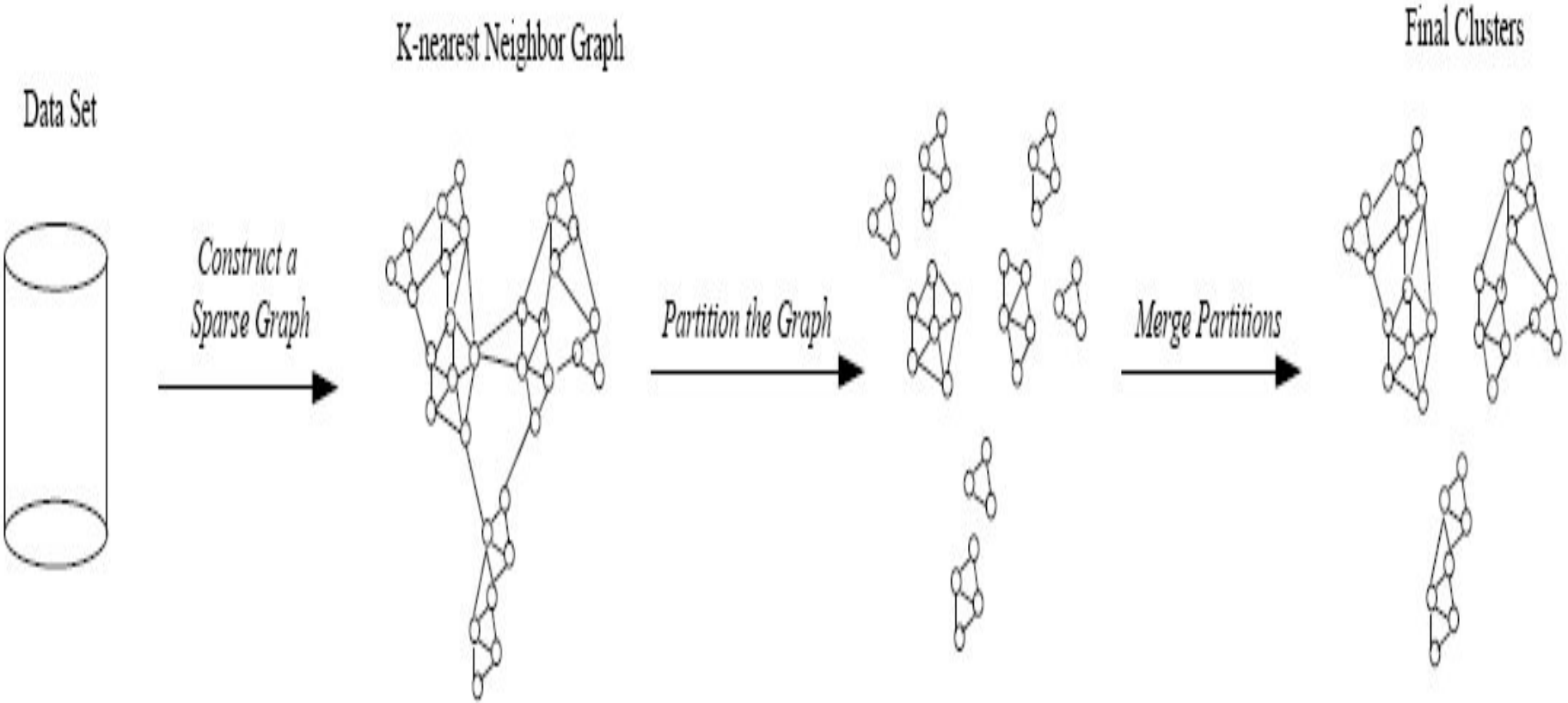
Step6. Assign each data into the closest cluster.

(3) CHAMELEON algorithm

- **A hierarchical Clustering Algorithm Using Dynamic Modeling**
- **Two-phase approach**
- **Phase –I (similarity between points)**
  - Uses a graph partitioning algorithm to divide the data set into a set of individual clusters.
- **Phase -II (similarity between clusters)**
  - uses an agglomerative hierarchical mining algorithm to merge the clusters.

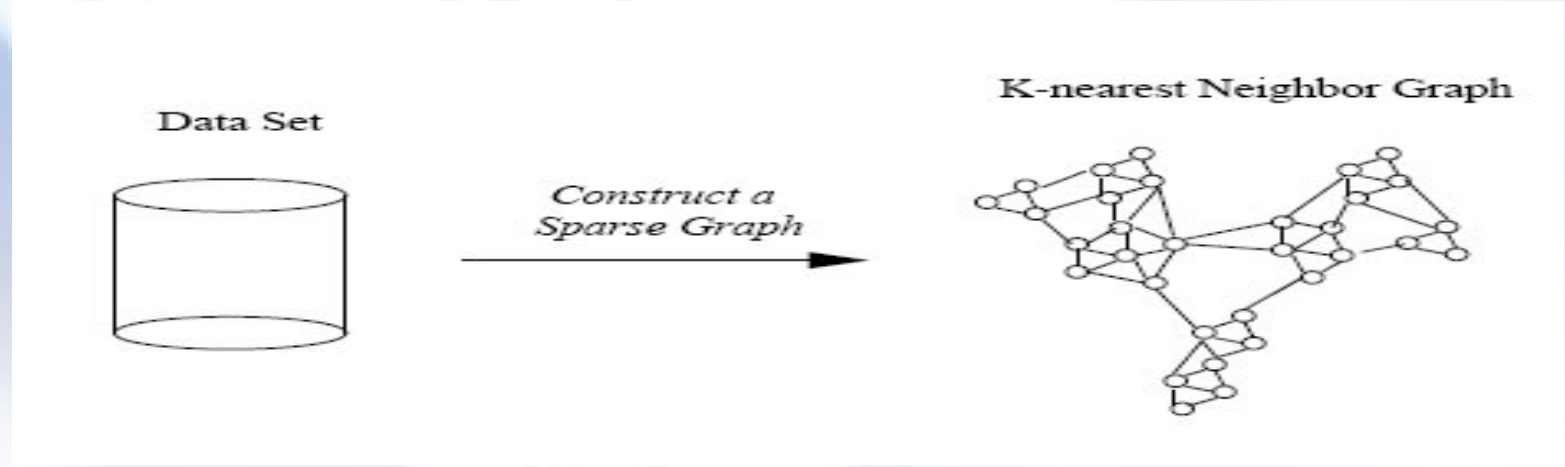
Graph based Method

CHAMELEON Steps



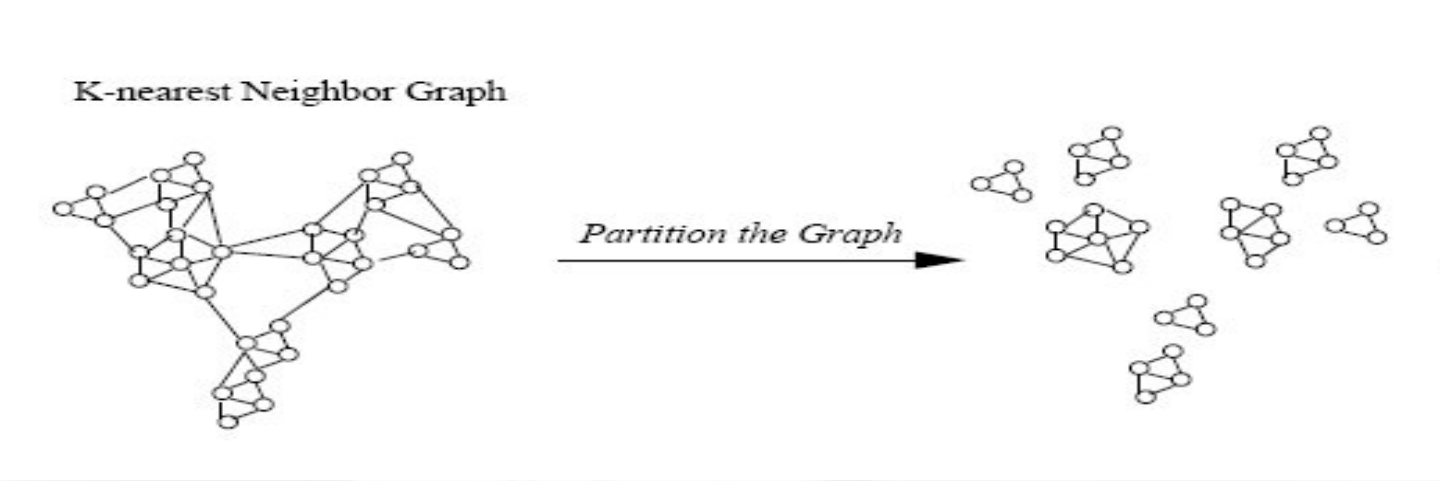
Costructing a sparse graph

- Using KNN
  - Data points that are far away are completely avoided by the algorithm (reducing the noise in the dataset)
  - captures the concept of neighbourhood dynamically by taking into account the density of the region.



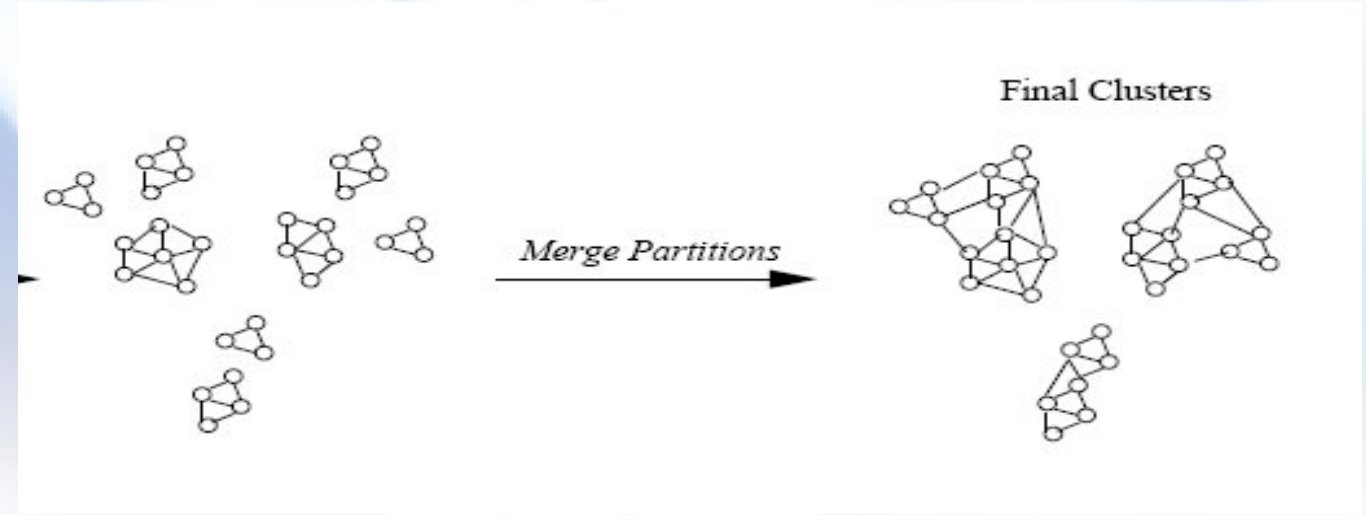
Graph Partitioning

- **Partition the KNN graph such that the edge cut is minimized.**
  - Reason: Since edge cut represents similarity between the points, less edge cut => less similarity.
- **Multi-level graph partitioning algorithms to partition the graph – hMeTiS library.**



Cluster Merging

- Takes into account features intrinsic to a cluster.
- Models cluster similarity based on the relative inter-connectivity (between two clusters) and relative closeness (within one cluster) of the clusters.

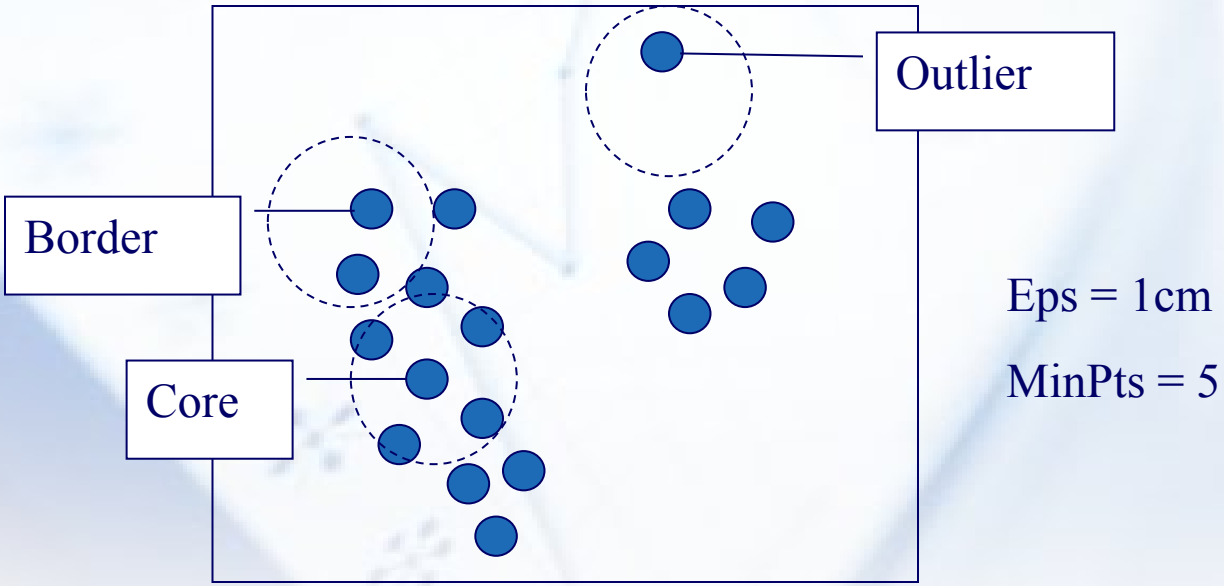


3.4 Density-Based Clustering

- **Clustering based on density (number of objects or data points) in the regions.**
- **Regard clusters as dense regions.**
- **Major features:**
  - Discover clusters of arbitrary shape
  - Handle noise
  - One scan
  - Need density parameters as termination condition
- **Representative algorithms:**
  - DBSCAN (Ester et al., 1996)
  - DENCLUE (Hinneburg & Keim, 1998)

DBSCAN

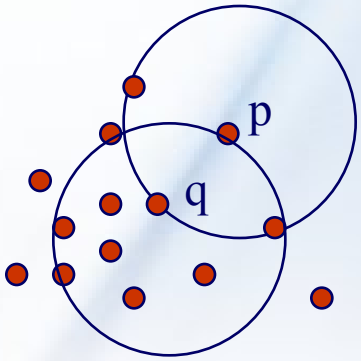
- **Density Based Spatial Clustering of Applications with Noise**
- **A *cluster* is defined as a maximal set of density-connected points**



How to define density?

- **Two density parameters:**
  - *Eps:* Maximum radius of neighborhood
  - *MinPts:* Minimum number of points in an Eps-neighborhood of a point
- ***Neighbor: $N_{Eps}(p) = \{q \in D \mid dist(p,q) <= Eps\}$***
- **Directly density-reachable: A point *p* is directly density-reachable from a point *q* wrt. *Eps*, *MinPts* iff**
  - 1) *p* belongs to $N_{Eps}(q)$
  - 2) *q* is a core point:

    $|N_{Eps}(q)| >= MinPts$

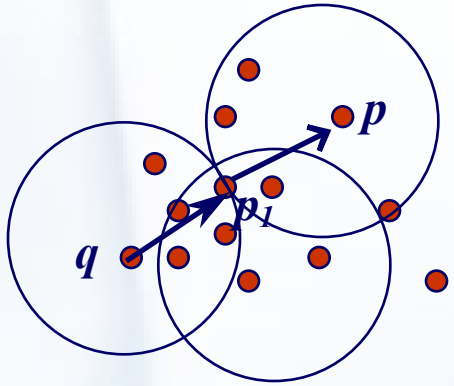


MinPts = 5

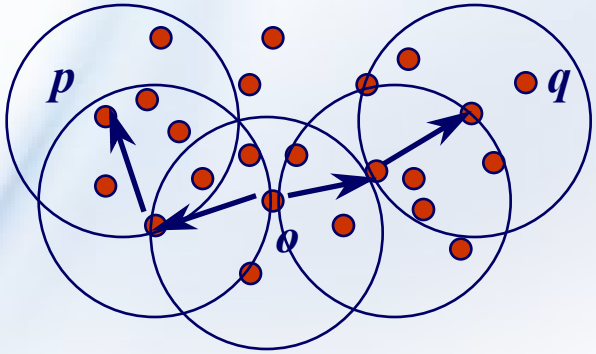
Eps = 1 cm

How to define density? contd.

- **Density-reachable:**
  - A point *p* is density-reachable from a point *q* wrt. *Eps*, *MinPts* if there is a chain of points p_1, …, p_n, p_1 = *q*, p_n = *p* such that p_{i+1} is directly density-reachable from p_i



- **Density-connected**
  - A point *p* is density-connected to a point *q* wrt. *Eps*, *MinPts* if there is a point *o* such that both, *p* and *q* are density-reachable from *o* wrt. *Eps* and *MinPts*.



DBSCAN algorithm steps

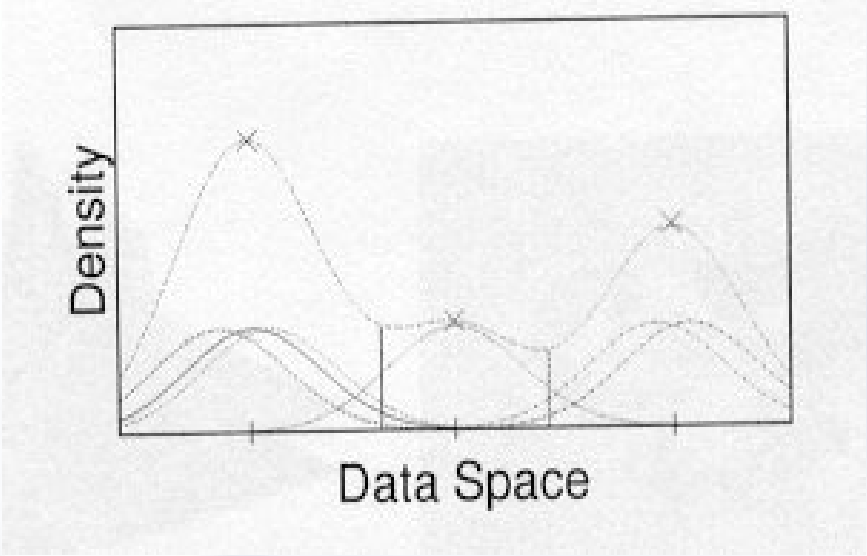
- Arbitrarily select a point *p*
- Retrieve all points density-reachable from *p* wrt *Eps* and *MinPts*.
- If *p* is a core point, a cluster is formed.
- If *p* is a border point, no points are density-reachable from *p* and DBSCAN visits the next point of the database.
- Continue the process until all of the points have been processed.

DENCLUE

- **DENsity-based CLUstEring (Hinneburg & Keim, 1998)**
- **Major features**
  - Good for data sets with large amounts of noise
  - Allows a compact mathematical description of arbitrarily shaped clusters in high-dimensional data sets
  - Significantly faster than other algorithms (faster than DBSCAN by a factor of up to 45)
  - But needs a large number of parameters

How to define the density?

- **Influence function**: describes the impact of a data point within its neighborhood.
- **Density function**: the sum of the influence function of all data points.
- **Density Attractor X***: Local maximum of the density function



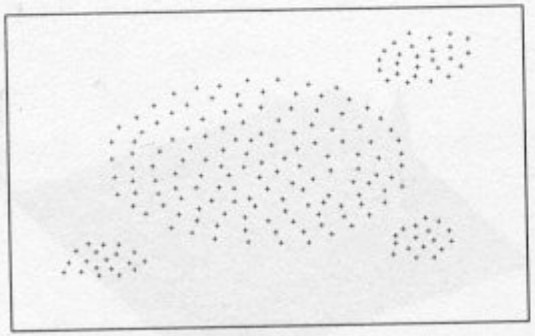
Examples of influence functions

1. Square Wave Influence Function

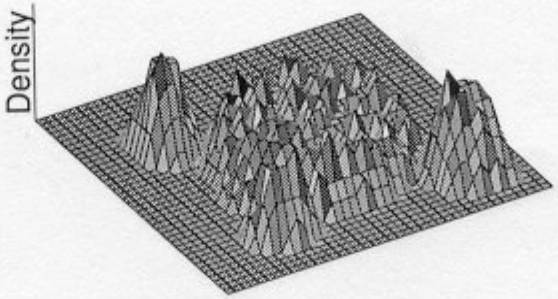
$$f_{Square}(x, y) = \begin{cases} 0 & \text{if } d(x, y) > \sigma \\ 1 & \text{otherwise} \end{cases}$$

2. Gaussian Influence Function

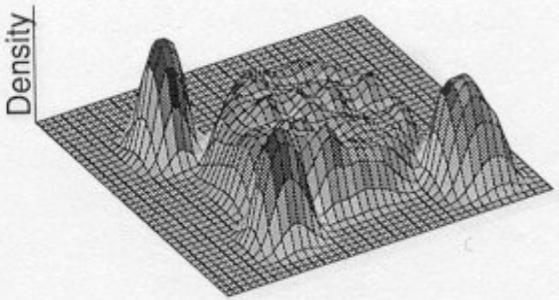
$$f_{Gauss}(x, y) = e^{-\frac{d(x,y)^2}{2\sigma^2}}$$



(a) Data Set



(b) Square Wave

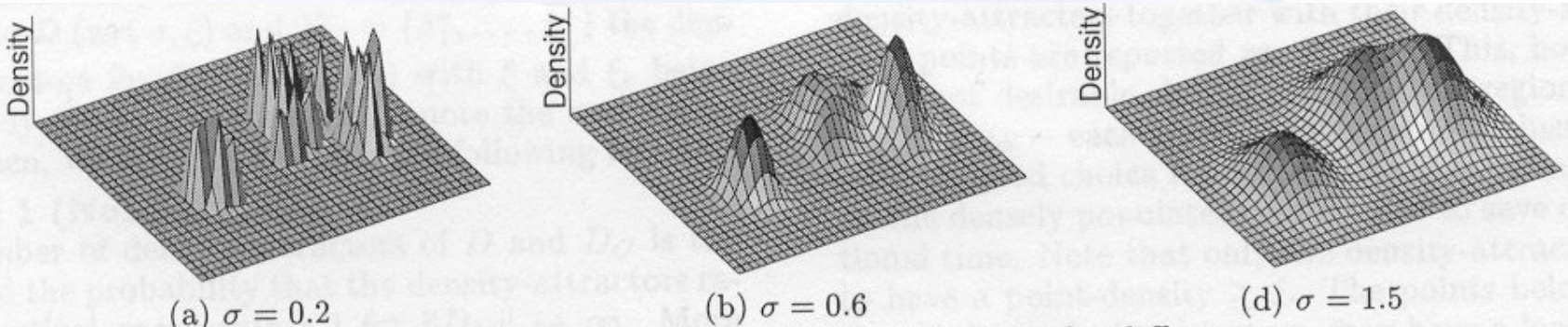


(c) Gaussian

Cluster Definition

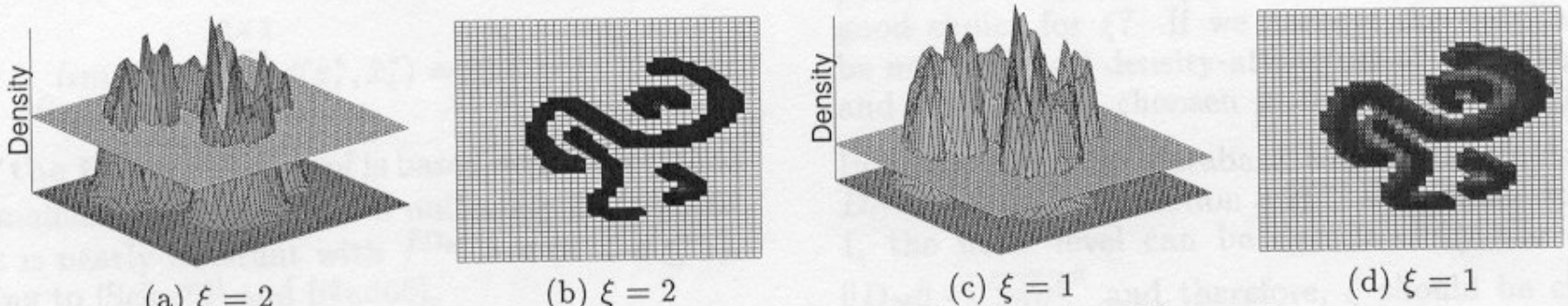
- **Center Defined Clusters**
  - All points that are density attracted to a given density attractor
  - Density function at the maximum must exceed a threshold
  - Points that are attracted to smaller maxima are considered outliers
- **Arbitrary-Shape Clusters**
  - Merges center defined clusters if a path exists for which the density function continuously exceeds a threshold

Examples of Clusters



(a) $\sigma = 0.2$ (b) $\sigma = 0.6$ (d) $\sigma = 1.5$

Figure 3: Example of Center-Defined Clusters for different σ



(a) $\xi = 2$ (b) $\xi = 2$ (c) $\xi = 1$ (d) $\xi = 1$

Figure 4: Example of Arbitray-Shape Clusters for different ξ

DENCLUE Steps1

- **Step 1: Construct a map of data points**
  - Uses hypercubes of with edge length 2s
  - Only populated cubes are saved
- **Step 2: Determine density attractors for all points using hill-climbing**
  - Keeps track of paths that have been taken and points close to them

DENCLUE Algorithm Step 1

Step 1: Construct a map of data points

- **Uses hypercubes of with edge length 2s,** Hypercubes contain:
  - Number of data points
  - Pointers to data points
  - Sum of data values (for mean)
- **Save populated hypercubes in B^+ tree**
- **Connect neighboring populated cubes for fast access**
  - Limited to highly populated cubes derived from outlier criterion

DENCLUE Algorithm Step 2

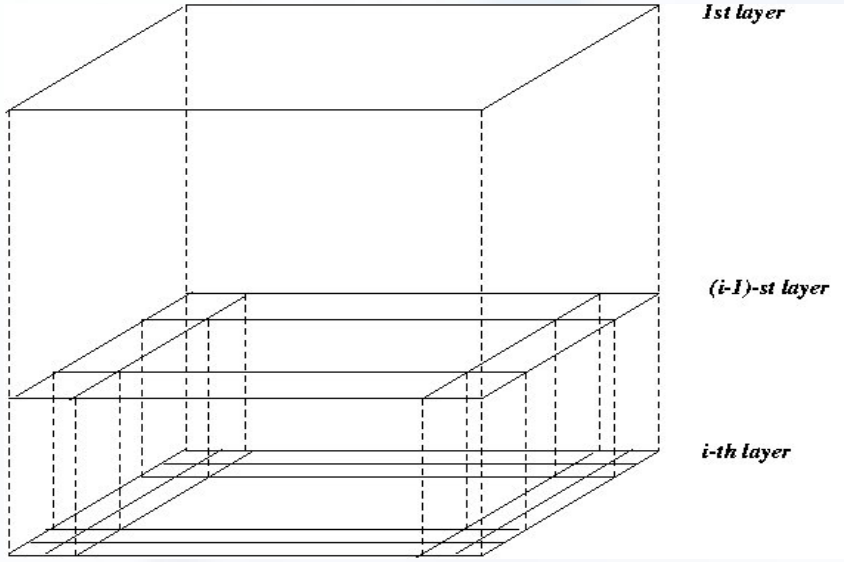
Step 2: Determine density attractors for all points using hill-climbing (clustering)

- Uses only highly populated cubes and cubes that are connected to them
- Hill-climbing based on local density function and its gradient
- Points within s/2 of each hill-climbing path are attached to clusters as well

3.5 Grid-based Clustering Methods

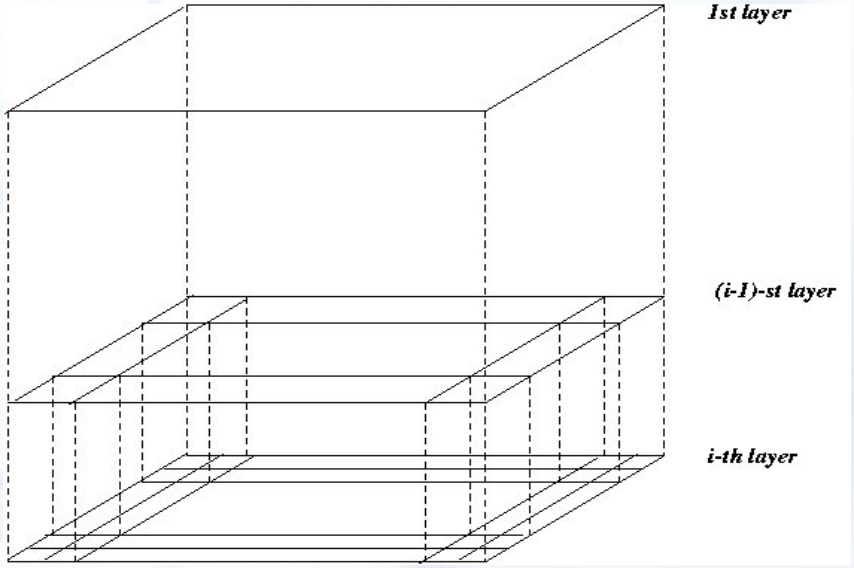
- **Ideas**
  - Divide the feature space into small cells (Grid)
  - Using multi-resolution grid data structures
  - Use dense grid cells to form clusters
- **Several interesting methods**
  - STING
  - WaveCluster
  - CLIQUE

Griding example:



STING

- A STatistical INformation Grid Approach
- The spatial area is divided into rectangular cells. There are several levels of cells corresponding to different levels of resolution

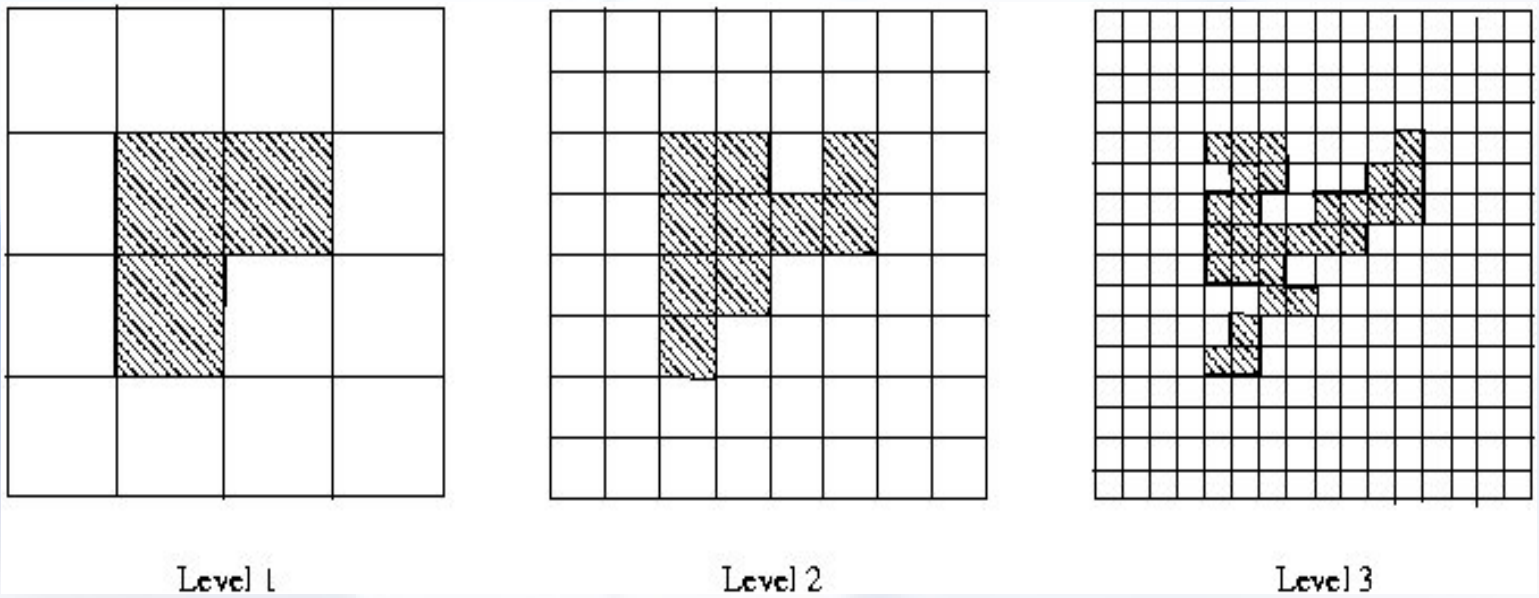


STING contd.

- **Each cell at a high level is partitioned into a number of smaller cells in the next lower level**
- **Parameters of higher level cells can be easily calculated from parameters of lower level cell**
  - *count*, *mean*, *standard deviation*, *min*, *max*
  - type of distribution—normal, *uniform*, etc.
- **Use a top-down approach to answer spatial data queries**
- **Start from a pre-selected layer—typically with a small number of cells**

STING contd.

- Remove the irrelevant cells from further consideration
- When finish examining the current layer, proceed to the next lower level
- Repeat this process until the bottom layer is reached



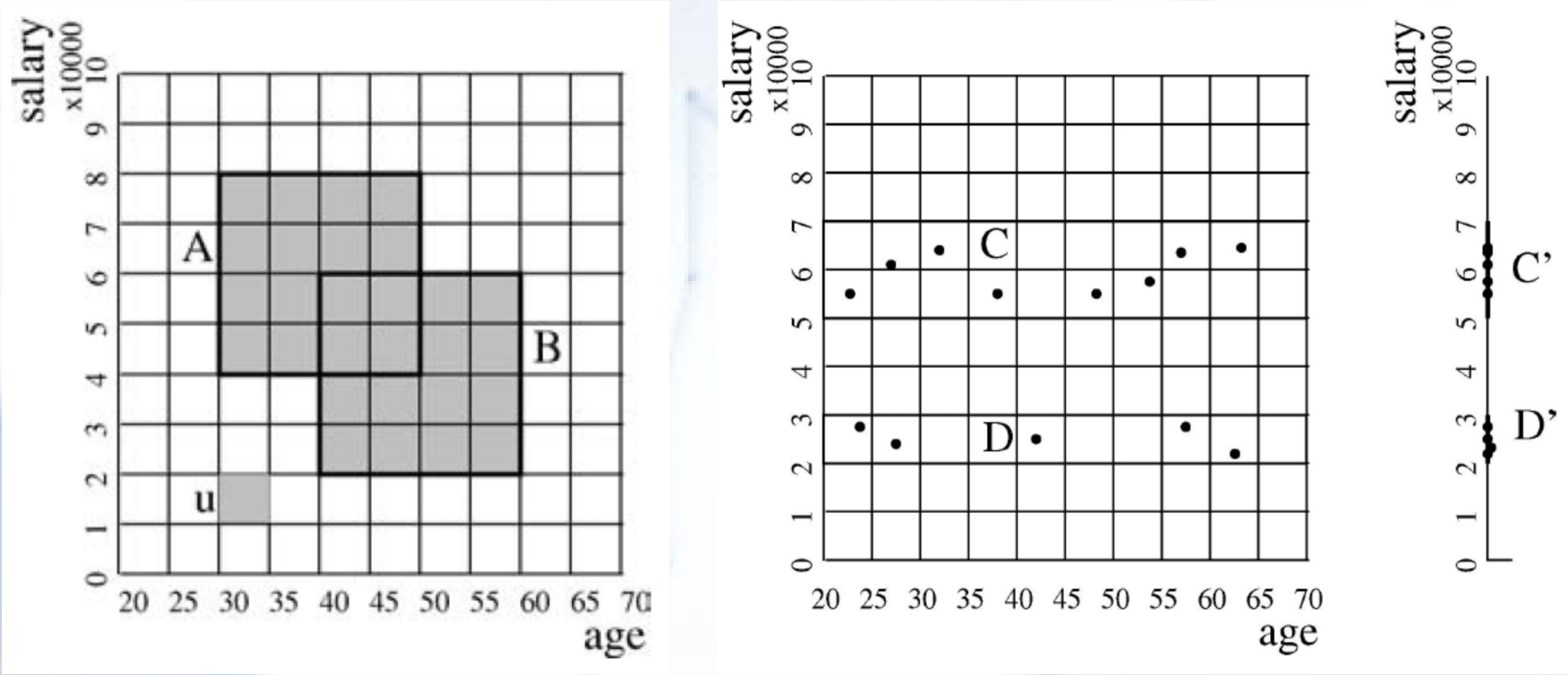
STING contd.

- **Advantages:**
  - Query-independent, easy to parallelize, incremental update
  - *O(K),* where *K* is the number of grid cells at the lowest level
- **Disadvantages:**
  - All the cluster boundaries are either horizontal or vertical, and no diagonal boundary is detected

CLIQUE

- CLustering In QUEst
- Automatically identifying subspaces of a high dimensional data space that allow better clustering than original space
- Both density-based and grid-based
  - It partitions each dimension into the same number of equal length interval
  - It partitions an m-dimensional data space into non-overlapping rectangular units
  - A unit is dense if the fraction of total data points contained in the unit exceeds the input model parameter
  - A cluster is a maximal set of connected dense units within a subspace

CLIQUE Example



clusters in original space clusters in subspace

CLIQUE: The Major Steps

- **Partition the data space and find the number of points that lie inside each cell of the partition.**
- **Identify the subspaces that contain clusters using the Apriori principle**
- **Identify clusters:**
  - Determine dense units in all subspaces of interests
  - Determine connected dense units in all subspaces of interests.
- **Generate minimal description for the clusters**
  - Determine maximal regions that cover a cluster of connected dense units for each cluster
  - Determination of minimal cover for each cluster

CLIQUE: Strength and Weakness

- **Strength**
  - It automatically finds subspaces of the highest dimensionality such that high density clusters exist in those subspaces
  - It is insensitive to the order of records in input and does not presume some canonical data distribution
  - It scales linearly with the size of input and has good scalability as the number of dimensions in the data increases
- **Weakness**
  - The accuracy of the clustering result may be degraded at the expense of simplicity of the method

Part 4: Unsupervised Learning: Association Rule Mining

"Beer and diaper" story

- **Beer and Diapers: The Impossible Correlation**



There are certain times at which beer and diapers sell particularly well together.

What is Association Rule？

- Association Rule (Association Rule Learning) is a rule-based machine learning method for discovering interesting relations between variables in large databases.
- It is intended to identify strong rules discovered in databases using some measures of interestingness.

What is Association Rule？

- Association rule mining was defined in 1993, when computer scientists Rakesh Agrawal, Tomasz Imieliński and Arun Swami developed an algorithm-based way to find relationships between items using point-of-sale (POS) systems.

- Association rule mining plays an important part in customer analytics, market basket analysis, product clustering, catalog design and store layout.

Support-Confidence Framework

Basic Concepts

- Let $I = \{i_1, i_2, \ldots, i_n\}$ be a set of *n* binary attributes called **items**.
- Let $D = \{t_1, t_2, \ldots, t_m\}$ be a set of **transactions** called the **database**.
- Each **transaction** in *D* has a unique transaction ID and contains a subset of the items in *I*: $T \subseteq I$
- **Example:**

Transaction

TID	Items
100	{beer, chips, wine}

Itemset

- **A collection of one or more items**
  - E.g. {beer, chips}
- **k-itemset**
  - An itemset that contains k items
  - E.g. A 3-itemset: {beer, chips, wine}

itemsets



TID	Items
100	{beer, chips, wine}
200	{beer, chips}
300	{pizza, wine}
400	{chips, wine}

Frequent Itemset

- **Support count(σ)**
  - Frequency of occurrence of an itemset
  - E.g. $\sigma(\{\text{beer, chips}\}) = 2$
- **Support**
  - Fraction of transactions that contain an itemset
  - E.g. $s(\{\text{beer, chips}\}) = 0.5$
- **Frequent Itemset**
  - An itemset whose support is greater than or equal to a *minsup* threshold

TID	Items
100	{beer, chips, wine}
200	{beer, chips}
300	{pizza, wine}
400	{chips, wine}

Association Rule

- An implication expression of the form X⇒Y, where X and Y are itemsets
- E.g. {beer, chips} ⇒ {wine}

TID	Items
100	{beer, chips, wine}
200	{beer, chips}
300	{pizza, wine}
400	{chips, wine}

Rule Evaluation Metrics

- **Support (s)**
  - Fraction of transactions that contain both X and Y
  - s (X ⇒ Y) = *P* (X ∪ Y)
- **Confidence (c)**
  - Measures how often items in Y appear in transactions that contain X
  - c (X ⇒ Y) = *P* (Y | X) = s (X ∪ Y) / s (X)

Rule Evaluation Metrics

- **E.g. {chips} ⇒ {beer}**
- **Support (s)**
  - $s = \frac{\sigma(\text{chips, beer})}{|T|} = \frac{2}{4} = 0.5$
- **Confidence (c)**
  - $c = \frac{\sigma(\text{chips, beer})}{\sigma(\text{chips})} = \frac{2}{3} = 0.67$

TID	Items
100	{beer, chips, wine}
200	{beer, chips}
300	{pizza, wine}
400	{chips, wine}

Association Rule Mining Task

- **Given a set of transactions T, the goal of association rule mining is to find all rules having:**
  - support ≥ *minsup* threshold
  - confidence ≥ *minconf* threshold
- **Two-step process of association rules mining:**
  - Find all frequent itemsets
  - Generate strong association rules from the frequent itemsets

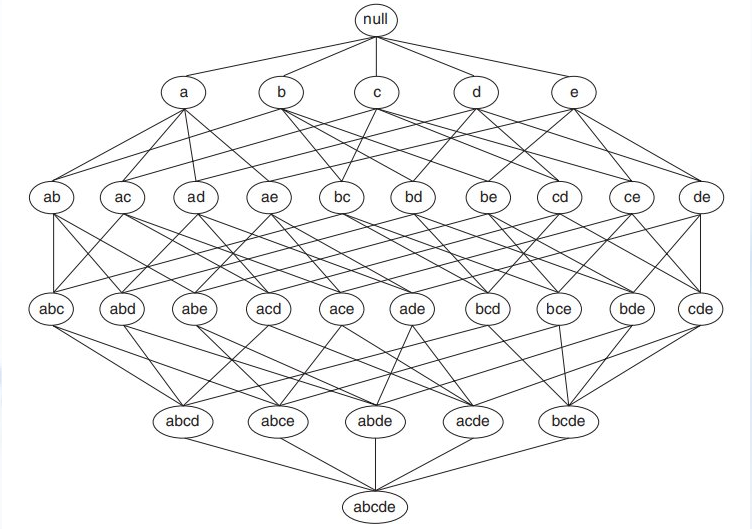
Frequent Itemset Generation

Frequent Itemset Generation

- Frequent itemset generation is computationally expensive
- Given d items, there are 2^d possible candidate itemsets

Frequent Itemset Generation

- A lattice structure can be used to enumerate the list of all possible itemsets.
- Each itemset in the lattice is a **candidate** frequent itemset

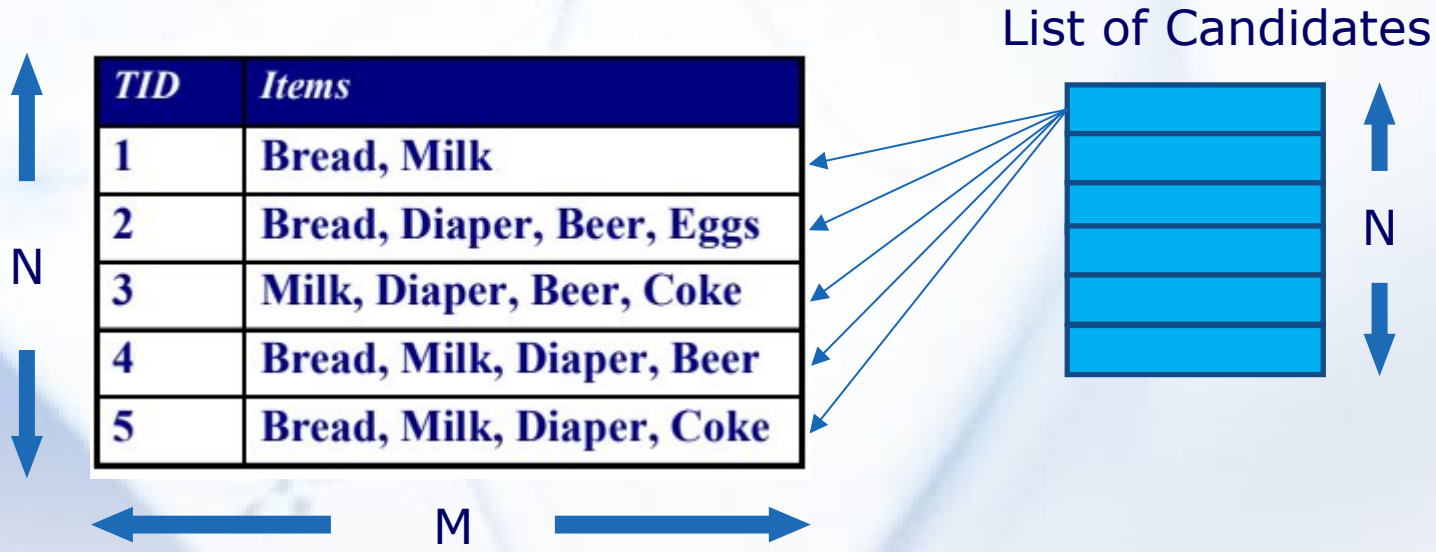


Brute-Force Approach

- A brute-force approach for finding frequent itemsets is to determine the support count for every candidate itemset, we need to compare each candidate against every transaction to determine the support count for every candidate itemset

Brute-Force Approach

- **Complexity: O(NMW)**
- **Expensive since M = 2^d**

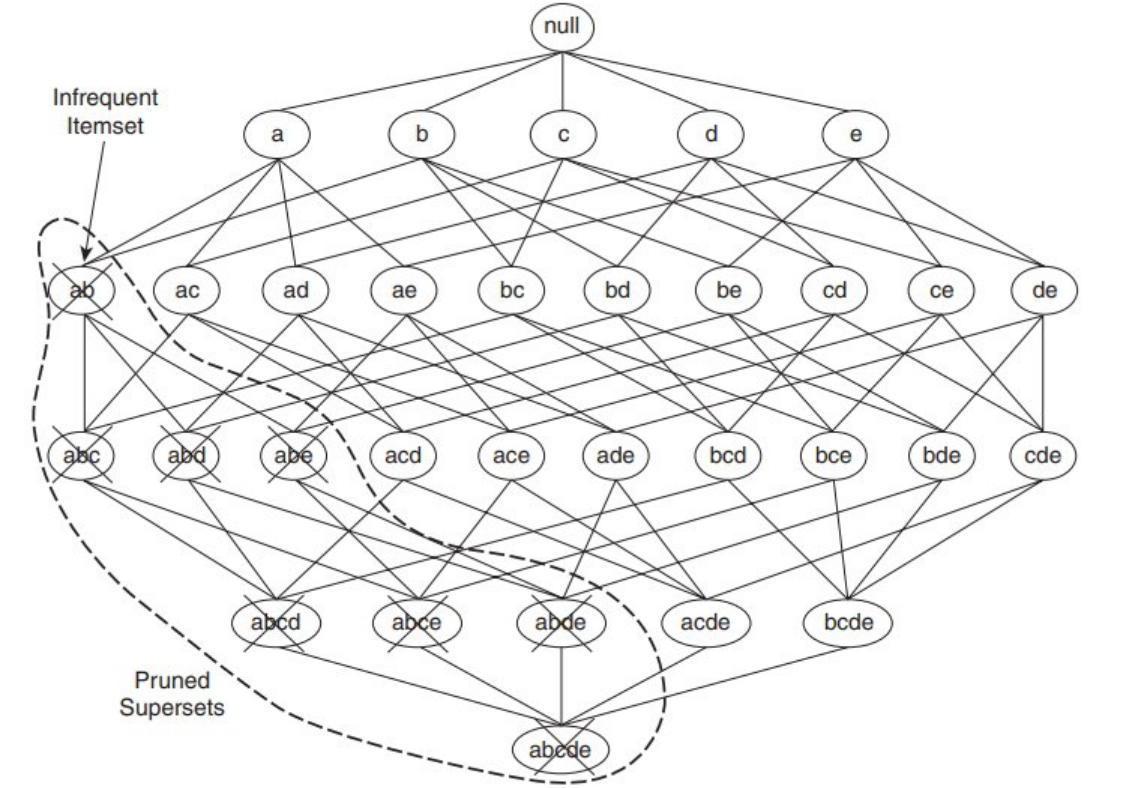


Apriori Algorithm

- Apriori is the first association rule mining algorithm that pioneered the use of support-based pruning to systematically control the exponential growth of candidate itemsets.
- If an itemset is frequent, then all of its subsets must also be frequent.

Apriori Algorithm

- E.g. itemset {a, b} is infrequent, all of its supersets must be infrequent too



Apriori Algorithm

- **Illustration of frequent itemset generation using the Apriori algorithm:**
  - Assume that the support threshold is 60%, which is equivalent to a minimum support count equal to 3
  - Initially, every item is considered as a candidate 1-itemset.

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke



Candidate 1-Itemsets

Item	Count
Beer	3
Bread	4
Cola	2
Diapers	4
Milk	4
Eggs	1

Apriori Algorithm

- the candidate itemsets {Cola} and {Eggs} are discarded because they appear in fewer than three transactions

removed because of low support

Candidate 1-Itemsets

Item	Count
Beer	3
Bread	4
Cola	2
Diapers	4
Milk	4
Eggs	1



Item	Count
Beer	3
Bread	4
Diapers	4
Milk	4

Apriori Algorithm

- In the next iteration, candidate 2-itemsets are generated using only the frequent 1-itemsets because the Apriori principle ensures that all supersets of the infrequent 1-itemsets must be infrequent

Item	Count
Beer	3
Bread	4
Diapers	4
Milk	4



Candidate 2-Itemsets

Itemset	Count
{Beer, Bread}	2
{Beer, Diapers}	3
{Beer, Milk}	2
{Bread, Diapers}	3
{Bread, Milk}	3
{Diapers, Milk}	3

Apriori Algorithm

- Two of these six candidates, {Beer, Bread} and {Beer, Milk}, are subsequently found to be infrequent after computing their support values.

removed because of low support

Candidate 2-Itemsets

Itemset	Count
{Beer, Bread}	2
{Beer, Diapers}	3
{Beer, Milk}	2
{Bread, Diapers}	3
{Bread, Milk}	3
{Diapers, Milk}	3



Itemset	Count
{Beer, Diapers}	3
{Bread, Diapers}	3
{Bread, Milk}	3
{Diapers, Milk}	3

Apriori Algorithm

- The remaining four candidates are frequent, and thus will be used to generate candidate 3-itemsets.

Itemset	Count
{Beer, Diapers}	3
{Bread, Diapers}	3
{Bread, Milk}	3
{Diapers, Milk}	3



Candidate 3-Itemsets

Itemset	Count
{Bread, Diapers, Milk}	2

Apriori Algorithm

- With the Apriori principle, we only need to keep candidate 3-itemsets whose subsets are frequent.
- The only candidate that has this property is {Bread, Diapers, Milk}. However, even though the subsets of {Bread, Diapers, Milk} are frequent, the itemset itself is not.

removed because of low support

Candidate 3-Itemsets

Itemset	Count
{Bread, Diapers, Milk}	2

Apriori Algorithm

- The algorithm initially makes a single pass over the data set to determine the support of each item. The set of all frequent 1-itemsets will be known
- Next, the algorithm will iteratively generate new candidate k-itemsets and prune unnecessary candidates that are guaranteed to be infrequent given the frequent (k - 1)-itemsets found in the previous iteration
- The algorithm terminates when there are no new frequent itemsets generated

Apriori Algorithm

Algorithm Frequent itemset generation of the *Apriori* algorithm.

1: $k = 1$.
2: $F_k = \{ i \mid i \in I \wedge \sigma(\{i\}) \geq N \times minsup \}$. {Find all frequent 1-itemsets}
3: **repeat**
4: $k = k + 1$.
5: $C_k = \text{candidate-gen}(F_{k-1})$. {Generate candidate itemsets.}
6: $C_k = \text{candidate-prune}(C_k, F_{k-1})$. {Prune candidate itemsets.}
7: **for** each transaction $t \in T$ **do**
8: $C_t = \text{subset}(C_k, t)$. {Identify all candidates that belong to t.}
9: **for** each candidate itemset $c \in C_t$ **do**
10: $\sigma(c) = \sigma(c) + 1$. {Increment support count.}
11: **end for**
12: **end for**
13: $F_k = \{ c \mid c \in C_k \wedge \sigma(c) \geq N \times minsup \}$. {Extract the frequent k-itemsets.}
14: **until** $F_k = \emptyset$
15: Result $= \bigcup F_k$.

Other algorithms

- **Eclat stands for Equivalence CLAss Transformation**
  - **uses a Depth-First Search approach**

- **FP-Growth Algorithm encodes the data set using a compact data structure called an FP-tree and generates frequent itemsets from an FP-tree by exploring the tree in a bottom-up fashion**

Rule Generation

Mining the Association Rules

- Given a frequent itemset {L}, find all non-empty subsets {f} ⊂ {L} such that the association rule {f} ⇒ {L – f} satisfies the minimum confidence.

- Create the rule {f} ⇒ {L - f}

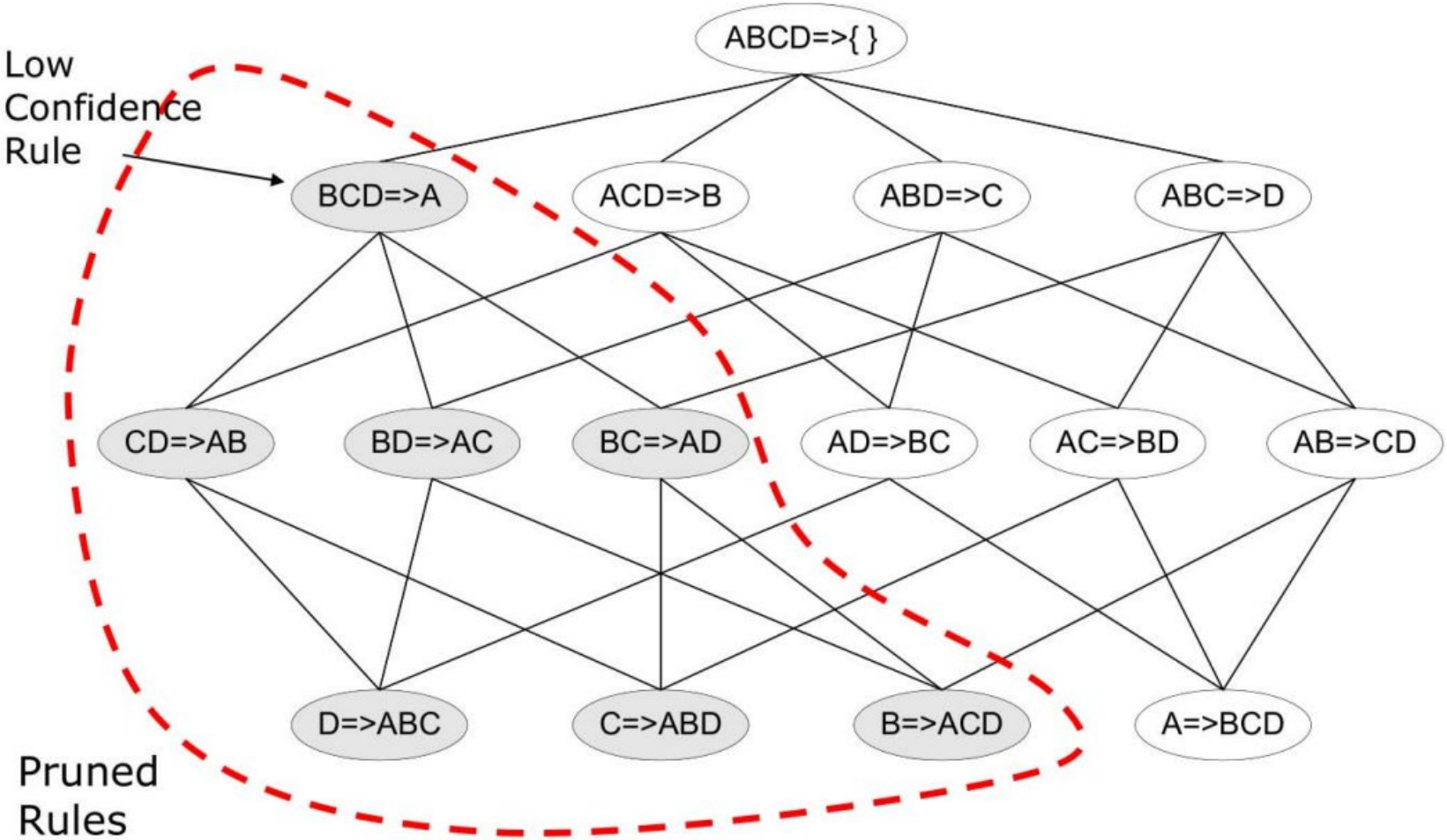
Rule generation

- E.g. A three item set will be partitioned to generate 6 rules:
  - An item set {A, B, C} generates rules
    - {A, B} ⇒ {C}
    - {A, C} ⇒ {B}
    - {B, C} ⇒ {A}
    - {A} ⇒ {B, C}
    - {B} ⇒ {A, C}
    - {C} ⇒ {A, B}
- If |L| = k, then there are 2k – 2 candidate association rules

Rule generation

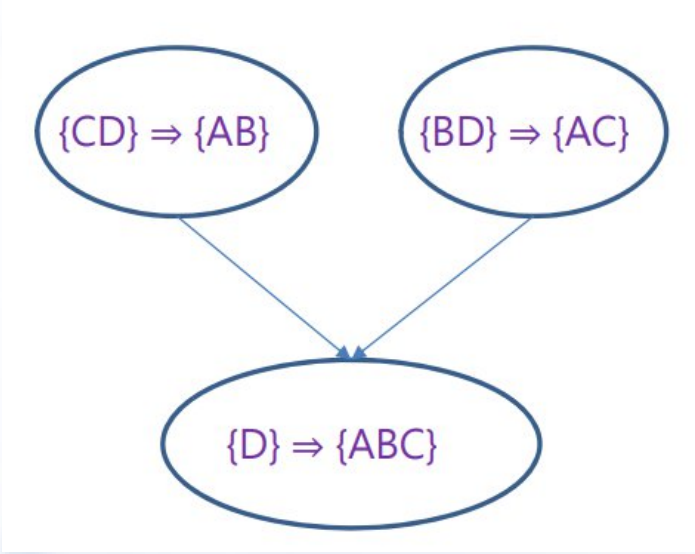
- confidence of rules generated from the same itemset has an anti-monotone property
  - E.g., Suppose {A, B, C, D} is a frequent 4-itemset:
  - conf({A, B, C} ⇒ {D}) ≥ conf({A, B} ⇒ {C, D}) ≥ conf({A} ⇒ {B, C, D})
- We can apply this property to prune the rule generation.

Rule generation



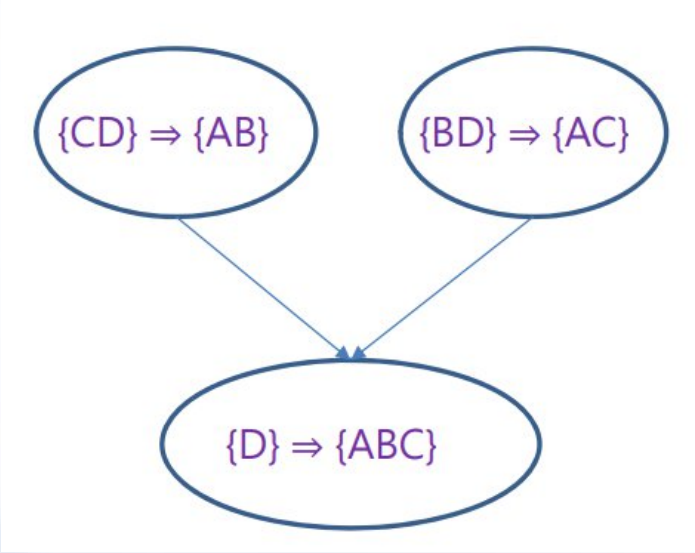
Rule generation

- Candidate rule is generated by merging two rules that share the same prefix in the rule consequent.



Rule generation

- Join({C, D} ⇒ {A, B}, {B D} ⇒ {A, C}) would produce the candidate rule {D} ⇒ {A, B, C}
- Prune rule {D} ⇒ {A, B, C} if its subset {A, D} ⇒ {B, C} does not have high confidence.



BIT.MLMR

End of Lecture 2

Have you grasp the main ideas and methods of supervised and unsupervised learning?

LIUXIABI@BIT.EDU.CN





